Community Experience Distilled

RavenDB High Performance

Learn how to accelerate your application development by building scalable applications on the RavenDB document database

Foreword by Ayende Rahien, Hibernating Rhinos

Brian Ritchie

[PACKT] PUBLISHING open source* community experience distilled

www.allitebooks.com

RavenDB High Performance

Learn how to accelerate your application development by building scalable applications on the RavenDB document database

Brian Ritchie

[PACKT] PUBLISHING
open source
community experience distilled

BIRMINGHAM - MUMBAI

www.allitebooks.com

RavenDB High Performance

Copyright © 2013 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: August 2013

Production Reference: 1210813

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78216-698-6

www.packtpub.com

Cover Image by Anna Kucherova (http://shutterstock.com)

www.allitebooks.com

Credits

Author
Brian Ritchie

Reviewers
Mohammed Ibrahim
Ayende Rahien
Paul Stovell

Acquisition Editors
Erol Staveley
Saleem Ahmed

Commissioning Editor
Poonam Jain

Technical Editors
Dipika Gaonkar
Sonali S. Vernekar

Project Coordinator
Esha Thakker

Proofreader
Chrystal Ding

Indexers
Tejal Soni
Priya Subramani

Production Coordinator
Manu Joseph

Cover Work
Manu Joseph

www.allitebooks.com

www.allitebooks.com

Foreword

About five years ago, I was working as a consultant dealing with a lot of "Strategic projects" for customers. Strategic project, I have learned, was a code name for something that costs a lot of money, and was usually in trouble.

At that time, I was dealing mostly with NHibernate (open source object relational mapper library), so I had a chance to see the type of issues people run into with their databases. In essence, those issues came in two different flavors. The first flavor was "let us try to shove a square through a round hole" in which the project was trying to do things that the poor RDBMS was just never meant to do. And the second flavor was "we are in a hurry, just get it over with" style.

A memorable occasion of running into that was the 15K project. I named it that way because every single page view would generate 15,000 database queries. And just to make sure that it is no mistake, yes, I am talking about fifteen thousand queries per page.

That is how RavenDB was born. I decided that I wanted to create a database that would make sense. A database you wouldn't have to fight with all day just to get where you want. RavenDB was designed to be a second generation database. That isn't just a slogan; it means that we have taken a lot of time to look at what else was around, to see what worked and where things didn't quite work.

Few of the things that we really tried were to go with: it just works, safe by default, and falling into the pit of success. That means, by default, RavenDB is going to try to do the right thing. But we also know that we can't figure one out on our own for all scenarios, so we also added a lot of knobs that you can tweak to make sure that RavenDB is doing exactly what you want it to do.

You can think about it like a game. You can come in and start playing, and everything is going to be really fun. But, there is also an entire extra layer that you can explore; this includes cheat codes, secret levels, and god mode.

www.allitebooks.com

That is what this book is meant to be. It is the cheat sheet that you can refer to get the absolute most from your RavenDB application. This is a book that is meant to take you from the merely good to the absolutely awesome. It explores a lot of the nooks and crannies in RavenDB; all the things that you need to know beyond the basics.

Brian has done a great job in not only explaining how things work, but also in selecting what are the sort of things that you need to know. The book covers issues such as scaling out (both sharding and replication), deploying to the cloud, multiple ways to extend RavenDB and customize it to your needs, monitoring in production, and a whole host of other things.

I would especially pay attention to the chapters about optimizing your data model and API, although I expect most people would go directly to the scale out sections. I have found that by making relatively small changes in the way I am modeling my data, I can get quite a bit more from my systems, without making the jump for a distributed system.

And after I actually move to a multi-node system, whether it is a highly available cluster or a set of sharded node (or indeed, the common mode of sharded and replicated cluster), the actual data model is probably the most important aspect for performance, scalability, and maintainability.

To summarize, go ahead and do the cool stuff first. But pay attention to the whole book. I think that Brian has done an excellent job in getting the crucial information you need to really get the most from RavenDB. And this book is one of the best ways to go from being merely proficient to being absolutely awesome.

Enjoy reading!

Oren Eini aka **Ayende Rahien**
Hibernating Rhinos

www.allitebooks.com

About the Author

Brian Ritchie is a software architect with a track record of developing large scale enterprise systems and leading development teams through difficult scalability challenges. Brian has 20 years of experience in Software Development and is currently Chief Architect at PaySpan, Inc. which provides innovative reimbursement solutions to healthcare payers and providers.

Brian is active in the development community giving presentations to local user groups and code camps. He has also contributed to Mono (the open source version of .NET) and various other open source projects. You can follow him on Twitter at `http://twitter.com/brian_ritchie` and read his blog at `http://weblogs.asp.net/britchie`.

Thanks to my wife, Michelle, whose love, support, and patience made this book possible. Also, thanks to my wonderful kids, Austin and Jordan, for understanding Dad's working late into the night. I would also like to recognize my Mom for instilling in me a love for learning and my high school computer science teacher, Don Kirkwood, for laying a great foundation with his unique style of teaching.

Enormous thanks also go to the Packt team for their help putting this book together and guiding me along the path. Finally, this book was improved greatly by the contributions of the excellent reviewers: Ayende Rahien, Mohammed Ibrahim, and Paul Stovell. I appreciate the time they took out of their busy schedules to share their insight and knowledge.

---

www.allitebooks.com

About the Reviewers

Mohammed Ibrahim is a .NET developer working for a Glasgow-based company Maclean Electrical for the past few years. He graduated from Glasgow Caledonian University. He has been developing web and windows applications using C# while implying agile methodologies, domain driven development, test driven development, continuous integration, and modern techniques. Mohammed has been developing projects using latest technologies, such as RavenDB, XUnit, MOQ, ASP.NET MVC, WPF, WCF, Automapper, PostSharp, NLog, KnockoutJS, JQuery. QUnit, HTML5, and so on.

I would like to thank my mother (Samira AL-Goubury).

Ayende Rahien (aka Oren Eini) has over 15 years of experience in the development world with a strong focus on the Microsoft and .NET ecosystem. He was awarded the Microsoft's Most Valuable Professional award in 2007. An internationally known presenter, he has taken conferences on DevTeach, JAOO, QCon, Oredev, NDC, Yow!, and Progressive.NET. He is the author of the book "DSLs in Boo: Domain Specific Languages in .NET", published by Manning (`http://manning.com/rahien/`). His main focus is on architecture and best practices that promote quality software and zero-friction development. He uses the pseudonym Ayende Rahien, and is a frequent blogger at `http://www.ayende.com/Blog/`.

He is the founder of Hibernating Rhinos LTD, which created RavenDB and is the project lead for the database.

www.allitebooks.com

Paul Stovell is the founder of Octopus Deploy, an automated deployment server designed for .NET developers. Prior to starting Octopus Deploy, he worked for an investment bank in London building WPF (Windows Presentation Foundation) applications, and before that for Readify, an Australian .NET consulting firm. He has also worked on a number of open source projects and has been a Microsoft MVP since 2006.

www.allitebooks.com

www.PacktPub.com

Support files, eBooks, discount offers and more

You might want to visit `www.PacktPub.com` for support files and downloads related to your book.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at `www.PacktPub.com` and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at `service@packtpub.com` for more details.

At `www.PacktPub.com`, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.

`http://PacktLib.PacktPub.com`

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can access, read and search across Packt's entire library of books.

Why Subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print and bookmark content
- On demand and accessible via web browser

Free Access for Packt account holders

If you have an account with Packt at `www.PacktPub.com`, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

Table of Contents

Preface 1

Chapter 1: A Different Kind of Database 5

- **Explosive growth** 5
- **Semi-structured data** 6
- **Architecture changes** 6
- **Rethinking the database** 7
  - Document databases 7
- **A document database for the .NET platform** 8
  - RavenDB architecture 9
    - Storing documents 9
    - Searching and retrieving documents 10
- **Summary** 11

Chapter 2: Optimizing the Data Model 13

- **Thinking in a document centric world** 13
  - A pattern-based approach to modeling relationships 14
    - One-to-one relationships 14
    - One-to-many relationships 14
    - Many-to-many relationships 15
- **Modeling relationships using keys** 16
- **Data modeling by example** 18
- **Efficiently accessing documents** 19
  - Working with indexes 21
  - Materialized views 23
  - Aggregating data using reduce 24
  - Efficiently loading related documents 24
- **Handling changing data over time** 25
- **Efficiently storing large chunks of data** 26
- **Summary** 28

Chapter 3: Optimizing the API 29
Optimizing large data loads 29
Reading large result sets 31
Being smart and lazy 32
Sometimes it is all about the little things 33
Patching meets scripting 36
Using patching to migrate document schema 37
Doing nothing is the fastest way to do something 38
Getting aggressive about doing nothing 39
Summary 40
Chapter 4: Monitoring and Tuning 41
Profiling isn't just for crime solvers 41
Profiling using Glimpse 43
Securing Glimpse 44
Monitoring with performance counters 45
Looking inside using server and database statistics 46
Looking under the hood 47
Tuning the server 48
Less I/O = more speed 48
Tuning the settings 50
Summary 50
Chapter 5: Scaling Out 51
Sharding 51
Sharding in the dark 52
Sharding based on the data 54
Sharding based on – whatever you like! 55
Scaling out reporting to a relational database 58
Summary 62
Chapter 6: When Failure is not an Option 63
Replication 63
Master-slave 64
Master-master 64
Multi-master 64
Configuring replication 65
Handling conflicts 67
Automatically handling conflicts 68
Handling node failures 68
Failover clustering 69
Read stripping 71
Summary 73

Chapter 7: Deploying to the Cloud **75**
Platform as a Service (PaaS) **76**
Infrastructure as a Service (IaaS) **76**
Database as a Service (DaaS) **76**
Getting started is easy 77
AppHarbor Integration 79
Summary **80**
Chapter 8: Extending RavenDB **81**
Triggers **81**
Customizing the storage format **84**
Server-side tasks **86**
Summary **87**
Chapter 9: Optimizing the User Experience **89**
Searching like Amazon **89**
Searching like Google **92**
Building responsive applications **96**
Displaying real-time updates 96
Summary **100**
Index **101**

Preface

RavenDB is an exciting document database built from the ground up on the .NET platform bringing unique capabilities to the .NET developer. A key goal of this book is to orient .NET enterprise and relational database developers with RavenDB and help you to think differently about data problems. RavenDB is not just another relational database. It is fundamentally different and requires a fresh perspective.

This book also approaches RavenDB from the perspective of scalability, availability, and performance. Taking RavenDB into big data environments requires learning to harness the power of the document data model and the RavenDB API.

What this book covers

Chapter 1, *A Different Kind of Database*, introduces you to the NoSQL movement, document databases, and why RavenDB is an important new tool for software engineers.

Chapter 2, *Optimizing the Data Model*, presents the concepts of document database modeling with an eye to building a high performance database.

Chapter 3, *Optimizing the API*, discusses specialized RavenDB APIs to tune database interactions.

Chapter 4, *Monitoring and Tuning*, covers server monitoring and aids in pinpointing bottlenecks and performance issues.

Chapter 5, *Scaling Out*, discusses options for distributing data across a multi-node cluster for ultimate scalability.

Chapter 6, *When Failure is not an Option*, discusses building a resilient data architecture which must be the foundation of operating a high performance system.

Chapter 7, *Deploying to the Cloud*, covers the options for deploying RavenDB into the cloud.

Chapter 8, *Extending RavenDB*, discusses high performance techniques for building distributed applications by extending the RavenDB server.

Chapter 9, *Optimizing the User Experience*, covers advanced capabilities for providing highly effective user experiences.

What you need for this book

In order to test and compile all of the examples in this book, you will need to install .NET 4.5 Framework, which is supported on all versions of Windows from Windows Vista and up, which you can find at `http://www.microsoft.com/en-us/download/details.aspx?id=30653`.

You will also need RavenDB 2.0 and in some cases 2.5 as noted. These versions are available for download from `http://ravendb.net/download`.

Who this book is for

RavenDB High Performance is a concise book covering advanced topics and assumes a basic understanding of RavenDB. It is written for developers building or planning large-scale systems with high performance requirements. The book will also benefit those who are designing systems in order to achieve high performance right from the start.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text are shown as follows: "We can include other contexts through the use of the `include` directive."

A block of code is set as follows:

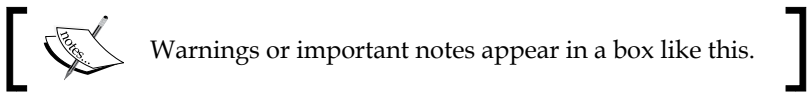
```
class Program
{
  static void Main(string[] args)
   {
```

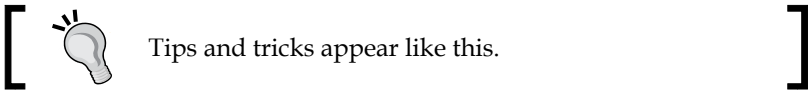
```
        Console.WriteLine("Hello World!");
      }
    }
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello World!");
    }
}
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "clicking the **Next** button moves you to the next screen".





Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book—what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to `feedback@packtpub.com`, and mention the book title via the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on `www.packtpub.com/authors`.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting `http://www.packtpub.com/submit-errata`, selecting your book, clicking on the **errata submission form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded on our website, or added to any list of existing errata, under the Errata section of that title. Any existing errata can be viewed by selecting your title from `http://www.packtpub.com/support`.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at `copyright@packtpub.com` with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at `questions@packtpub.com` if you are having a problem with any aspect of the book, and we will do our best to address it.

1
A Different Kind of Database

When most people talk of a database, they mean a relational database. Relational databases have been the foundation of enterprise application for the past 30 years. First defined in June 1970 by *Edgar Codd* of IBM's San Jose Research Laboratory, relational databases store data in now familiar tables made up of rows and columns.

Relational databases have served us well for many years, so why do we need a different kind of database? Most developers have experience of building applications with relational databases and access to great tooling. However, relational databases do have their limits. As our systems grow, it becomes more difficult and expensive to scale a traditional relational database.

To understand how we got here, let's take a quick trip back into the recent past. Relational databases were created when big iron ruled the world. These centralized mainframes provided the foundation of the first relational database systems. As we moved into the client/server era, these databases moved onto lower priced servers. But fundamentally, they are still running on one central machine.

Explosive growth

Relational databases worked well when systems were serving hundreds or even thousands of users, but the Internet has changed all of that. The number of users and volume of data is growing exponentially. A variety of social applications have proved that applications can quickly attract millions of users. Relational databases were never built to handle this level of concurrent access.

www.allitebooks.com

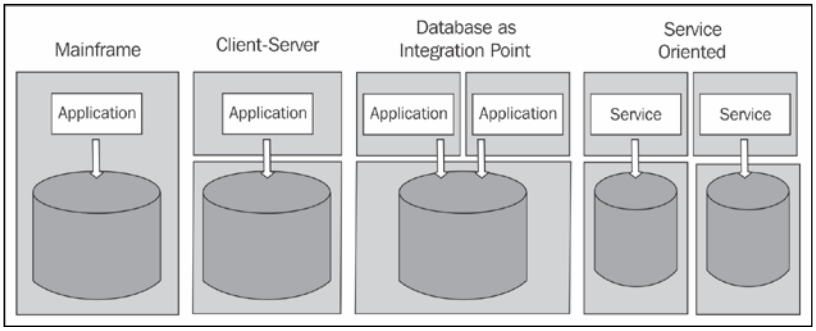
Semi-structured data

In addition to the staggering growth, data is no longer simple rows and columns. Semi-structured data is everywhere. **Extensible Markup Language** (**XML**) and **JavaScript Object Notation** (**JSON**) are the lingua franca of our distributed applications. These formats allow complex relationships to be modeled through hierarchy and nesting. Relational databases struggle to effectively represent these data patterns. Due to this impedance mismatch, our applications are littered with additional complexity. **Object relational mapping** (**ORM**) tools have helped but not solved this problem.

With the growth of **Software as a Service** (**SaaS**) and cloud-based applications, the need for flexible schemas has increased. Each tenant is hosted on a unified infrastructure but they must retain the flexibility to customize their data model to meet their unique business needs. In these multi-tenant environments, a rigid schema structure imposed by a relational database does not work.

Architecture changes

While data is still king, how we architect our data-dependent systems has changed significantly over the past few decades. In many systems, the database acted as the integration point for different parts of the application. This required the data to be stored in a uniform way since the database was acting as a form of API. The following diagram shows the architectural transitions:



With the move to **Service Oriented Architectures** (**SOA**), how data is stored for a given component has become less important. The application interfaces with the service, not the database. The application has a dependency on the service contract, not on the database schema. This shift has opened up the possibilities to store data based on the needs of the service.

Rethinking the database

The factors we have been discussing have led many in our industry to rethink the idea of a database. Engineers wrestled with the limitations of the relational database and set out to build modern web-scale databases. The term **NoSQL** was coined to label this category of databases. Originally, the term stood for No SQL but has evolved to mean Not Only SQL. To confuse matters further, some NoSQL databases support a form of the SQL dialect. However, in all cases they are not relational databases.

While the NoSQL landscape continues to expand with more projects and companies getting in the action, there are four basic categories that databases fall into:

- Document (CouchDB, MongoDB, RavenDB)
- Graph (Neo4J, Sones)
- Key/Value (Cassandra, SimpleDB, Dynamo, Voldemort)
- Tabular/Wide Column (BigTable, Apache Hbase)

Document databases

Document databases are made up of semi-structure and schema free data structures known as documents. In this case, the term document is not speaking of a PDF or Word document. Rather, it refers to a rich data structure that can represent related data from the simple to the complex. In document databases, documents are usually represented in JavaScript Object Notation (JSON). A document can contain any number of fields of any length. Fields can also contain multiple pieces of data. Each document is independent and contains all of the data elements required by the entity.

The following is an example of a simple document:

```
{
  Name: "Alexander Graham Bell",
  BornIn: "Edinburgh, United Kingdom",
  Spouse: "Mabel Gardiner Hubbard"
}
```

And the following is an example of a more complex document:

```
{
  Name: "Galileo Galilei",
  BornIn: "Pisa, Italy",
```

```
    YearBorn: "1564",
    Children: [
  { Name: "Virginia", YearBorn: "1600" },
  { Name: "Vincenzo", YearBorn: "1606" }
  ]}
```

Since documents are JSON-based, the impedance mismatch that exists between the object-oriented and relational database worlds is gone. An object graph is simply serialized into JSON for storage. Now, the complexity of the entity has a small impact on the performance. Entire object graphs can be read and written in one database operation. There is no need to perform a series of select statements or create complex stored procedures to read the related objects.

JSON documents also add flexibility due to their schema free design. This allows for evolving systems without forcing the existing data to be restructured. The schema free nature simplifies data structure evolution and customization. However, care must be given to the evolving data structure. If the evolution is a breaking change, documents must be migrated or additional intelligence needs to be built into the application.

A document database for the .NET platform

Prior to RavenDB, document databases such as CouchDB treated .NET as an afterthought. In 2010, *Oren Eini* from Hibernating Rhinos decided to bring a powerful document database to the .NET ecosystem. According to his blog:

> *Raven is an OSS (with a commercial option) document database for the .NET/ Windows platform. While there are other document databases around, such as CouchDB or MongoDB, there really isn't anything that a .NET developer can pick up and use without a significant amount of friction. Those projects are excellent in what they do, but they aren't targeting the .NET ecosystem.*

RavenDB is built to be a first-class citizen on the .NET platform offering developers the ability to easily extend and embed the database in their applications. A few of the key features that make RavenDB compelling to .NET developers are as follows:

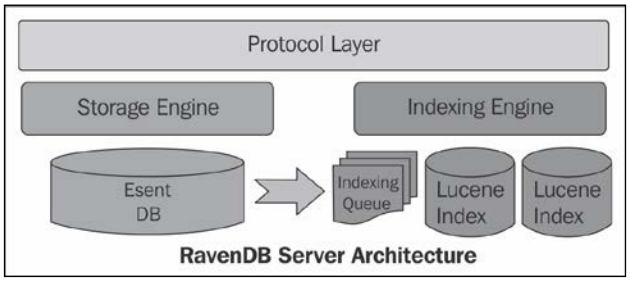
- RavenDB comes with a fully functional .NET client API, which implements unit of work, change tracking, read and write optimizations, and much more. It also has a REST-based API, so you can access it via the JavaScript directly.

- It allows developers to define indexes using **LINQ** (**Language Integrated Queries**). Supports map/reduce operations on top of your documents using LINQ.
- It supports `System.Transactions` and can take part in distributed transactions.
- The server can be easily extended by adding a custom .NET assembly.

RavenDB architecture

RavenDB leverages existing storage infrastructure called ESENT that is known to scale to amazing sizes. ESENT is the storage engine utilized by Microsoft Exchange and Active Directory. The storage engine provides the transactional data store for the documents. RavenDB also utilizes another proven technology called Lucene.NET for its high-speed indexing engine. Lucene.NET is an open source Apache project used to power applications such as AutoDesk, StackOverflow, Orchard, Umbraco, and many more.

The following diagram shows the primary components of the RavenDB architecture:



Storing documents

When a document is inserted or updated, RavenDB performs the following:

1. A document change comes in and is stored in ESENT. Documents are immediately available to load by ID, but won't appear in searches until they are indexed.
2. Asynchronous indexing task takes work from the queue and updates the Lucene index. The index can be created manually or dynamically based on the queries executed by the application.
3. The document now appears in queries. Typically, index updates have an average latency of 20 milliseconds. RavenDB provides an API to wait for updates to be indexed if needed.

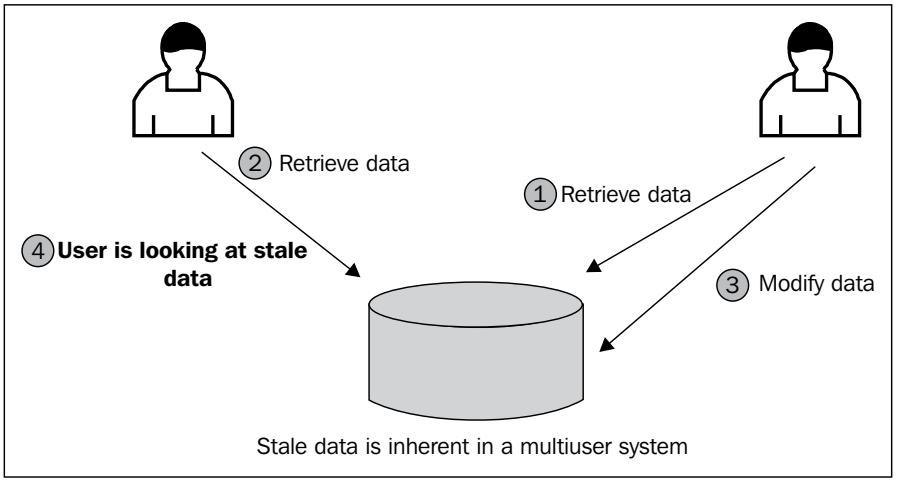
Searching and retrieving documents

When a document request comes in, the server is able to pull them directly from the RavenDB database when a document ID is provided. All searches and other inquiries hit the Lucene index. These methods provide near instant access, regardless of the database size.

A key difference between RavenDB and a relational database is the way index consistency is handled. A relational database ties index updates to data modifications. The insert, update, or delete only completes once the indexes have been updated. This provides users a consistent view of the data but can quickly degrade when the system is under heavy load.

RavenDB on the other hand uses a model for indexes known as **eventual consistency**. Indexes are updated asynchronously from the document storage. This means that the visibility of a change within an index is not always available immediately after the document is written. By queuing the indexing operation on a background thread, the system is able to continue servicing reads while the indexing operation catches up. Eventual consistency is a bit counter-intuitive. We do not want the user to view stale data. However, in a multiuser system our users view stale data all the time. Once the data is displayed on the screen, it becomes stale and may have been modified by another user.

The following diagram illustrates stale data in a multiuser system:



Stale data is inherent in a multiuser system

In many cases, this staleness does not matter. Consider a blog post. When you publish a new article, does it really matter if the article becomes visible to the entire world that nanosecond? Will users on the Internet really know if it wasn't? What typically matters is providing feedback to the user who made the change. Either let them know when the change becomes available or pausing briefly while the indexing catches up. If a user did not initiate the data change, then it is even easier. The change will simply become available when it enters the index. This provides a mechanism to give each user personal consistency. The user making the change can wait for their own changes to take affect while other users don't need to wait.

Eventual consistency is a tradeoff between application responsiveness for all users and consistency between indexes and documents. When used appropriately, this tradeoff becomes a tool for increasing the scalability of a system.

Summary

As you can see, RavenDB is truly a different kind of database. It makes fundamental changes to what we expect from a database. It requires us to approach problems from a fresh perspective. It requires us to think differently. Over the following chapters, we will explore how these design differences and unique capabilities help us build high performance applications.

2
Optimizing the Data Model

Most of us come from a relational database background and often our minds continue to solve problems in that context. Taking this approach in a document database world would be absolutely the wrong thing to do. While it is possible to model relational entities as documents, this would have significant performance and complexity implications. RavenDB is optimized for reading documents with nested object graphs, rather than reading multiple small documents. Due to this, a high performance database requires a well-designed document model.

Thinking in a document centric world

Document modeling requires looking at normalization in a different way. Instead of designing to third normal form (the default RDBMS route), the considerations for document modeling revolve around embedding or referencing.

Embedding puts related pieces of information together in the same document. Having data located in the same document improves performance and simplifies access. Data that is in a parent-child relationship is a great candidate for embedding. Unlike in relational modeling, embedding is the first choice for documents.

Referencing, on the other hand, separates data by storing a reference in one document to another. Referencing leads to more reads but provides additional flexibility. Only storing the key in the document prevents data duplication. Referencing is also optimal when modeling very large data sets or modeling many-to-many relationships. References typically point to the current associated document. If you need the reference to data at a given point in time, you will need to version the referenced document.

A good approach to determine when to embed or reference is to think in aggregates. **Aggregates** are a concept from **Domain Driven Design** (**DDD)**. Aggregates are groups of objects that belong together. For example, invoice lines have no reason to exist without an invoice. They also cannot belong to any other invoice. Invoice and invoice lines define an aggregate. In DDD, the primary container for an aggregate is called the **aggregate root**. When modeling a document, the aggregate becomes the document and it is named after the aggregate root.

A pattern-based approach to modeling relationships

RavenDB does not directly support entity relationships. However, relationships exist in the real world and these patterns will help you follow best practices when modeling them.

One-to-one relationships

In one-to-one relationships, the subordinate data should be embedded in the primary document. An example is as follows:

```
// person/1
{
  "name": "Sam Smith",
  "address" : {
    "street": "1720 Center Blvd",
    "city": "Jacksonville",
    "state": "FL",
    "postal": "32256"
    }
}
```

One-to-many relationships

In one-to-many relationships, a collection is embedded in the parent document representing many children. If items in the collection only make sense in the context of the parent, they should be embedded. If the items in the collections are aggregates, the collection should contain references. An example is as follows:

```
// manufacturers/1
{
  "name": "Toyota",
```

```
    "models": [
        { "name":"Prius" },
        { "name":"Camry" },
        { "name":"Highlander} ]
}
```

Be careful when embedding collections that may become large. When the collection reaches a certain size, you should either split the document or use the referencing technique.

Here is an example of splitting the document:

```
// manufacturers/1
{
  "name": "Toyota",
}

// manufacturers/1/models
{
  "models": [
      { "name":"Prius" },
      { "name":"Camry" },
      { "name":"Highlander} ]
}
```

By splitting the document, the models collection can be loaded only when needed, instead of loading every time the manufacture document is loaded.

Many-to-many relationships

In many-to-many relationships, one document maintains a list of document identifiers for the related documents. An example is as follows:

```
// books/1
{
  "title": "CLR via C#",
  "ISBN": "978-0735667457",
  "published": 2012
}

// books/2
{
  "title": "Applied Microsoft .NET Framework Programming",
  "ISBN": "978-0735614222",
  "published": 2002
}
```


www.allitebooks.com

```
// authors/1
{
  "name": "Jeffrey Richter",
  "books": [ { "id":"books/1", "title":" CLR via C#"},
             { "id":"books/2", "title":" Applied Microsoft .NET
             Framework Programming"} ]
}

// authors/2
{
  "name": "Maarten van de Bospoort",
  "books": ["id":"books/1", "title":" CLR via C#"}]
}
```

Note that the books collection not only contains the ID reference, but also the title. By adding this denormalization, the display of the author and related books can be optimized. When building an application, look for these opportunities to limit the need for multiple database reads.

Modeling relationships using keys

RavenDB is based on a RESTful architecture and as such supports meaningful URLs. In the preceding simple example, we have embedded the document (entity) and key into the URL. However, we can represent more complex relationships using the URL definition. For example, if we want to store each chapter with a book independently we could use the following scheme:

```
books/1
books/1/chapters/1
books/1/chapters/2
books/1/appendixes/1
books/1/appendixes/2
```

The `books`, `chapters`, and `appendixes` are each stored as a separate document. This allows efficient loading of a document directly by ID. Using the `LoadStartingWith()` method, you can also return all the chapters at once:

```
session.Advanced.LoadStartingWith<chapter>("books/1/chapters", null,
1, 10);
```

You can even use this to match a part of a complex key. For example, if your key contained phone numbers, you could use the following to load all of the phones in the 904 area code:

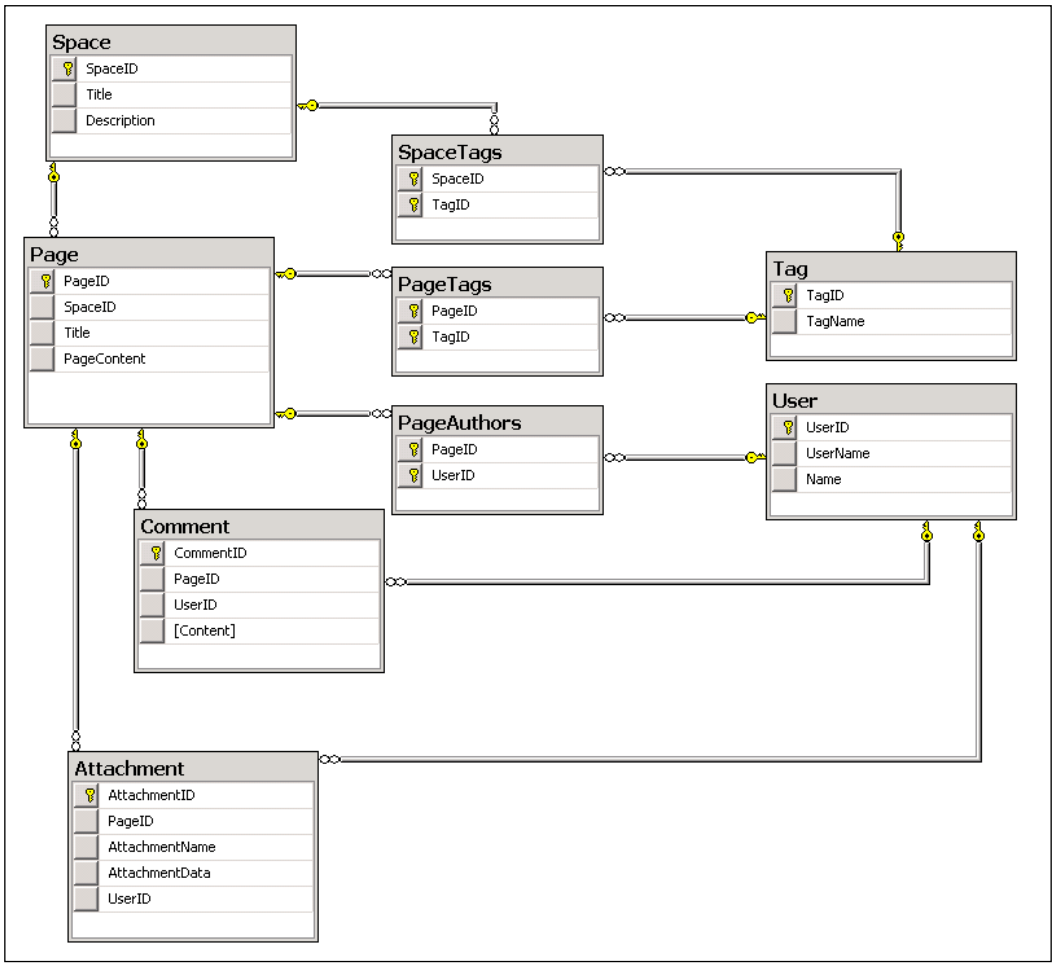
```
session.Advanced.LoadStartingWith<Phone>("phones/FL/904", null, 1,
10);
```

The second parameter in the `LoadStartingWith()` method supports additional filtering by passing a pipe "|" delimited list of wildcards. RavenDB supports "*" for matching multiple characters and "?" for matching on a single character just like searching a file system.

While RavenDB supports other methods to retrieve lists of documents, using keys is the fastest method since RavenDB can go directly to the data store to load the documents. The wildcard option is a bit more expensive since the documents are filtered on the server side after they are loaded from the data store. Another benefit of retrieving by key is that eventual consistency issues are avoided since the data store is consistent immediately after writes.

Data modeling by example

To better understand document modeling, let's walk through an example. We are designing a Wiki that consists of spaces, pages, comments, attachments, and tags. In a relational world, it would be modeled like the following:



But, we are not in a relational world. So, let's look at this from a document perspective. Which entities in the preceding model above are aggregates? Users, spaces, and pages each have the right to exist independently. Tags, comments, and attachments aren't really meaningful outside the context of their page.

In a document database, the data could be modeled as follows (we'll address attachments later):

```
// spaces/1
{
  "Title": "Development Team",
  "Description": "Standards and Architecture Development Team",
     "Tags": ["Team"]
}

// pages/1
{
  "Space": "spaces/1",
  "Title": "MVC Architecture",
  "Content": "... content ...",
  "Authors": ["users/sjones", "users/psmith"],
  "Tags": ["Architecture", "Standards"],
  "Comments": [
    {"Text": "Great points!!",
      "CreatedBy":"users/sjones" }
  ]
}

// users/sjones
{
  "Username": "sjones",
  "Name": "Sam Jones",
}
```

Efficiently accessing documents

When a document is loaded from RavenDB, all of the data is retrieved at once. In many cases, this is extremely optimal. For example, when we need to render the invoice in our hypothetical accounting system or a page in our Wiki, it only requires one request to the database. In the relational model, it would have taken multiple queries or joins that duplicate data.

The fastest way to load a document is to retrieve it by key. When retrieving by key, RavenDB can go directly to the data store and pull the document out quickly. Your data model should be designed to match the key used within the application code to the key used in the database. For example, in an ASP.NET MVC application the URL route can correspond directly to the key in the database.

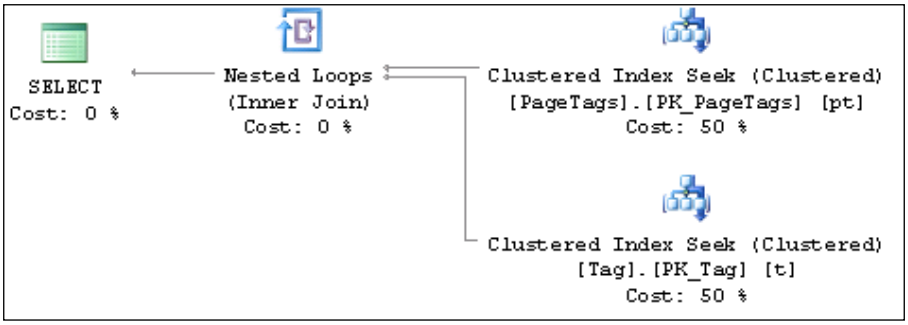
A URL like `http://localhost/page/1` would have a controller like the following:

```
public ActionResult Page(int id)
{
using (var session = documentStore.OpenSession())
   {
    var page = session.Load<Page>(id);
    return View(page);
   }
}
```

Instead of a single read, to perform the same retrieval using SQL Server it would have required the following commands:

```
SELECT p.Title,p.Content
FROM [Page] p
WHERE p.PageID = @PageID

SELECT t.TagName
FROM [PageTags] pt
INNER JOIN Tag t ON pt.TagID=t.TagID
WHERE pt.PageID = @PageID

SELECT u.UserName
FROM [PageAuthors] pa
INNER JOIN [User] u ON pa.UserID=u.UserID
WHERE pt.PageID = @PageID

SELECT c.Content, u.UserName
FROM [Comment] c
INNER JOIN [User] u ON c.UserID=u.UserID
WHERE c.PageID=@PageID
```

Depending on your implementation, this would have required multiple trips to the server. It would also require SQL Server to perform lots of work on every page load. Just to perform, one of the preceding `JOINS` looks like the following:



Multiply this query by three and add the query that loads the page content, and you start to get the picture. Also, if your content field is over 8 KB, the `VARCHAR(MAX)` method used to store the page content, will overflow to additional storage pages requiring SQL Server to perform additional reads.

While the `Load()` method is very efficient, we are limited to loading documents when we know the ID. If there is another field that is also unique and more descriptive, RavenDB can be configured to load documents by this field as well. To do this, install the Unique Constraints bundle by placing the `Raven.Bundles.UniqueContraints` assembly in the `Plugins` directory and referencing `Raven.Client.UniqueConstraints` within your application.

Add attributes to the document's properties that are unique constraints. The document will need to be stored after this attribute has been added to create the appropriate keys in the database.

```
class Page
{
    public string Id { get; set; }

    [UniqueConstraint]
    public string Title { get; set; }

    public string Content { get; set; }

    public string[] Tags { get; set; }
}
```

Next, register the `UniqueConstraintsStoreListener` listener:

```
store.RegisterListener(new UniqueConstraintsStoreListener());
```

Now the `LoadByUniqueConstraint` extension method can be used to load the document using the page title:

```
var page = session.LoadByUniqueConstraint<Page>(x => x.Title, "MVC
Architecture");
```

Working with indexes

Combining multiple entities into a single document has potential complications. In our invoice example, what if we wanted to know all of the products purchased on invoices in our system?

RavenDB provides a solution called **indexes**. Indexes have two purposes: querying and aggregation. Indexes are defined using LINQ queries. Under the hood, they are stored in Lucene, which gives RavenDB nearly instant access to data stored in an index. For aggregation, RavenDB uses map/reduce, a technique popularized by Google to efficiently process data across multiple threads. Indexes can either be created by the developer or dynamically generated by RavenDB when a query is executed.

A simple map index allows efficient querying of relevant parts of a document. A map is a transformation that extracts the pieces of data we are interested in. The following example allows quick searching by state:

```
from doc in docs.Customers
select new { doc.State }
```

When indexing runs, RavenDB extracts data from your documents using the map LINQ query. The results are used to create fields on a Lucene document. The Lucene fields become available to be queried against through the index. RavenDB also stores a field called `__document_id` that contains the key of your original document. When a query result is returned, RavenDB uses the `__document_id` field to load the original documents and return them as the result of your query.

Just like indexes in a relational database, a RavenDB index requires both storage and processing time when data is written. Creating too many indexes will slow down indexing writes and create excessive overhead on your server. A recommended approach is to use RavenDB's dynamic indexing feature. When a query is performed that does not have an index defined, RavenDB will generate a **dynamic index** automatically. This avoids the need to hand roll simple indexes and allows the RavenDB query optimizer to build the best index for the job. It also prevents duplicate indexes from being created when the optimizer finds your index inefficient and then generates another index of its own.

Care should be taken when deploying dynamic indexes on large production systems. A dynamic index must be built on the fly and will have indeterminate impact on application performance if the index is built when the feature is first accessed by a user. The dynamic indexes should be generated in a test environment and then deployed prior to the application. By applying the indexes as a deployment step, you can allow the indexer to finish its work before making the updated application live.

Materialized views

As previously discussed, a standard map index returns documents as the result of a query. If the documents in our database are large, performance will suffer if we bring back the entire document or referenced documents to simply display a grid of results.

For optimal performance we need a materialized view of just the necessary fields. RavenDB 2.5 introduces a result transformer that allows us to process the transformation server-side and return only the data we need. A transformer is defined by a special class that defines the operation to perform on the results through a special LINQ query. The query can access related documents to build a new virtual view containing the pieces we need.

The following example returns information from the page and space documents plus the comment count to display in a page list:

```
public class PageTransformer :
AbstractTransformerCreationTask<PageViewModel>
{
    public PageTransformer()
    {
        TransformResults = pages =>
                from page in pages
                select new
                {
                   page.Title,
                   CommentCount=page.Comments.Count,
                   SpaceName=LoadDocument<Space>(page.Space).Name
                };
    }
}
```

By only retrieving the count of comments, the data returned from the server is greatly reduced since a page could have dozens or even hundreds of comments. Also, the name field from the related space document is included into the result.

Now, it is simple to retrieve our transformed results in a single pass:

```
var pages = session.Query<Page>()
    .TransformWith<PageTransformer, PageViewModel>()
    .ToList();
```

Aggregating data using reduce

The **reduce** step adds another level of power to an index. It is used to aggregate data. In the reduce step, we can do things like group by, sum, count, and more. The beauty of indexes is that our groupings and aggregates are pre-calculated. We don't incur a performance penalty every time we query our index. Even better, the index is automatically updated when documents are added or changed.

To create an index to build a tag cloud, first the map operation extracts the necessary data:

```
from doc in docs.Pages
select tag in doc.Tags
select new { Tag = tag, Count = 1}
```

Next, the reduce operation aggregates the results:

```
from result in results
group result by result.Tag into g
select new { Tag = g.Key, Count = g.Sum(x=>x.Count) }
```

Now a simple query in C# can retrieve the results:

```
var tagCloud = docDb.Query<TagAndCount>("TagCloud");
```

Efficiently loading related documents

While documents are mostly self-contained, they may have references to other documents. Consider the page document has references to the user and space documents. In our Wiki example, the author attribute has the ID of the referenced user document.

Using the standard document loading syntax, it requires two accesses to the server to retrieve the author of the page:

```
var page = session.Load<Page>("pages/1");
var user = session.Load<User>(page.Author);
```

RavenDB client API provides the `Include()` function to solve this problem:

```
var page = session.Include<Page>(x => x.Author).Load("pages/1");

// this will not require querying the server!
var author = session.Load<User>(page.Author);
```

By using include, both documents are returned in a single HTTP response. The RavenDB session then keeps track of the documents you have loaded and checks its cache before going to the server on the second load request.

While this capability is powerful, it can be abused. If you find yourself repeating this throughout your code, you may want to reconsider your document data model. It might make more sense to copy certain attributes into the document. This is especially true if they rarely change. For example, if we just need the username when accessing the page document, it should be an attribute of the page.

Related documents may also need to be queried at the same time. For example, we may want to query all of the pages created by a user that belongs to a particular group. To do this, create an index using the `LoadDocument` command:

```
public class PageIndex : AbstractIndexCreationTask<Page>
{
    public PageIndex()
    {
       Map = pages =>
          from page in pages
          select new
          {
             Title = page.Title,
             AuthorGroup =
             LoadDocument<User>(page.Author).Group
          };
    }
  }
```

Handling changing data over time

So far, we have discussed documents that represent the current value of our data. But in many cases, data changes over time. How do we know what the previous state of the data was before it was updated? In relational databases, we often create history tables to store these old versions. Fortunately, RavenDB has a much cleaner approach. It has a Versioning bundle that adds support for automatically storing snapshots of documents every time they are changed.

The Versioning bundle will copy the historical documents into a location relative to the document. For example, if we have a document called `pages/1`, then older versions would be stored at `pages/1/revisions/1`, `pages/1/revisions/2`, and so on. You can learn more at about the Versioning bundle at the RavenDB web site: `http://ravendb.net/docs/2.5/server/extending/bundles/versioning`.

In the preceding example discussed, only one page is active at a time: the current page. However, there are times when data is effective for a period of time. An example of this is product pricing. The product price changes over time and may go on sale in the future. This pricing scheme needs to be set up in advance and then have the system make the correct decision based on the current date.

A third-party bundle called Temporal Versioning adds this capability. You can download it via NuGet or get the source code here: `https://github.com/mj1856/RavenDB-TemporalVersioning`. Once we have the bundle in place, we can easily deal with temporal data. To save a document with an effective date, simply do the following:

```
session.Effective(dto).Store(Product);
session.SaveChanges();
```

Loading documents is equally easy:

```
// To load current data, use standard syntax:
var foo = session.Load<Foo>("products/1");

// To load past or future data, specify the effective date
var foo = session.Effective(dto).Load<Product>("products/1");
```

As you can see, these bundles make dealing with data that change over time very easy. What is painful in a relational world is a natural extension of the document database. Of course, I have just touched on the basics of what is possible with these extensions so go online to learn more.

Efficiently storing large chunks of data

While RavenDB is not a general purpose file system, it can efficiently store large data chunks. It is recommended to store files outside the database, but there are situations where RavenDB makes sense. But before we explore the right way, let's look at the wrong way first. Here is a blog post where we just embedded a byte array:

```
public class Page
{
  public string Id { get; set; }
  public string Title { get; set; }
  public byte[] Picture { get; set; }
}
```

This will "work" and RavenDB will serialize the byte array using Base64. The problem is that this will bloat our document and every time we retrieve the document we will return the picture as well. So what is the right way? Use attachments! RavenDB contains a separate API for working with attachments. Each attachment has a unique ID and is accessed separately from documents but can be referenced by documents:

```
public class Page
{
  public string Id { get; set; }
  public string Title { get; set; }
  public string Picture { get; set; }
}
```

In this case, the `Picture` property would store the reference ID of the attachment. To store an attachment in RavenDB, use the `PutAttachment` command. The ID can be anything you want:

```
var db = documentStore.DatabaseCommands;
var metaData = new RavenJObject();
metaData["Format"] = "PNG";
using (var stream = File.OpenRead("admin-icon.png")))
{
   db.PutAttachment("pictures/12", null, stream, metaData);
}
```

Unlike files, RavenDB supports optional metadata, which is a handy feature. To retrieve the attachment and its associated metadata, use the `GetAttachment` command:

```
var db = documentStore.DatabaseCommands;
var attachement = db.GetAttachment("pictures/12");
var pic = attachement.Data();
var format = attachement.Metadata["Format"];
File.WriteAllBytes(@"temp.png", pic);
```

Besides simplicity, caching, and metadata, there is another reason to like attachments. RavenDB includes a Cascade Delete bundle to automatically delete an attachment associated with a document. To use the bundle, simply put the bundle DLL in the `plugins` directory.

```
using (var session = documentStore.OpenSession())
{
session.Store(doc);
session.GetMetadataFor(doc)
   ["Raven-Cascade-Delete-Attachments"] =
     new[] {" pictures/12"};
session.SaveChanges();
}
```

One point of caution: Cascade Delete only works within a single node. If you use sharding, cascading will not delete attachments located on other nodes.

Summary

The design of your document data model is a key to performance and simplicity. But it requires thinking from a document perspective and not a relational one. Your documents should represent the natural boundaries within your application. The other tools such as indexes are there to aid in querying and are not meant to replace good document design. Next up, we will move beyond the session-based APIs covered in this chapter. These specialized APIs allow for further performance optimizations.

3
Optimizing the API

Like any well-designed system, RavenDB's session-based API is designed to make the often used tasks simple. However, there are certain tasks that are better suited for RavenDB's specialized APIs.

In this chapter we will cover:

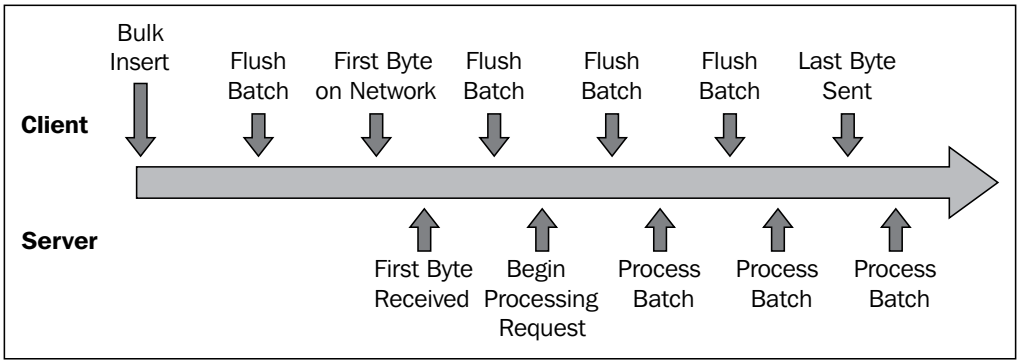
- Optimizing large data loads
- Processing large result sets
- Getting multiple results at once
- Patching documents instead of replacing them
- Built-in caching capabilities

Optimizing large data loads

When loading large amounts of data, there is a certain overhead with the standard API that becomes expensive. RavenDB 2.0 introduced a bulk insert API to solve this problem. It is usually orders of magnitude faster than sending batches through the standard API. This optimized approach does come with a few limitations that we will cover later in the chapter. Here is an example of loading an array containing a large set of page documents:

```
using (var bulkInsert = documentStore.BulkInsert())
{
  foreach (var page in PagesToInsertArray)
  {
    bulkInsert.Store(page);
  }
}
```

The bulk insert command not only optimizes the processing on the server, it also optimizes sending the data over the network. Instead of sending data on each call to the `Store()` method, data is transmitted in batches. These batches are processed on the server, concurrent to the next batch being prepared on the client. The default batch size is 512 documents. The following diagram shows how this process works between the client and the server:



Besides concurrent processing, batching also makes compression over the wire more efficient and allows the server to minimize the number of disk flushes when writing the data.

Like most bulk insert APIs, there are some limitations as follows:

- The entity ID must be provided by the client. When using the .NET client library this is not a problem since this happens automatically using the HiLo algorithm.
- Transactions are per batch, not for the entire operation. This means that an error in one of the documents will result in the entire batch of documents being rolled back, but will not roll back the previously committed batches. By default the batch is of 512 documents, but this is configurable. Also, **Distributed Transaction Coordiator** (**DTC**) transactions are not supported.
- Inserts will not raise notifications through the changes API.
- Document updates and reference checking must be explicitly turned on through the `BulkInsertOptions` parameter. Both of these options will impact the performance so only turn them on if needed. Document updates check to see if a document needs to be updated, otherwise, an update would fail the operation. Reference checking looks for documents referenced from indexing using `LoadDocument`.
- Put triggers will execute, but the AfterCommit triggers will not.

To sum it up, bulk inserts are very fast but come at the cost of the limitations listed in the preceding bullet list. Like Uncle Ben said in Spiderman: "With great power comes great responsibility", so be careful!

Reading large result sets

RavenDB was built to be safe by default. This means that it protects the client (and the server) from overflowing memory by reading large query results into memory. Therefore, all queries utilize paging to read results in chunks. If a page size value is not specified, the client API will limit the results to 128 documents. The server side also enforces its own hard limit to the page size of 1,024 results. Be aware that the server side limit is configurable but will always take precedence over the client page size.

For web applications, these limits are rarely a problem. In fact, most web-based grids use paging on the client side so using a standard LINQ query fits perfectly. Let's take a quick look at using the built-in paging support through a sample MVC controller action. The following action uses the standard `Skip()` and `Take()` LINQ methods to return one page worth of data. Since our view needs to display the total number of pages, we use the `Statistics()` method to return the total items. Now we can efficiently page through a virtually unlimited result set:

```
public class ListResultModel<T>
{
    public List<T> Results { get; set; }
    public int PageIndex { get; set; }
    public int ItemsPerPage { get; set; }
    public int TotalItems { get; set; }
}

public ActionResult Index(int PageIndex, int ItemsPerPage)
{
    RavenQueryStatistics stats;
    var result = session.Query<Page>()
                    .Statistics(out stats)
                    .Skip((PageIndex - 1) * ItemsPerPage)
                    .Take(ItemsPerPage)
                    .ToList();
    return View(new ListResultModel<Page>
                {
                    Results = result,
                    ItemsPerPage = ItemsPerPage,
                    PageIndex = PageIndex,
                    TotalItems = stats.TotalResults
                }
        );
}
```

But, there are times when this behavior has performance penalties. For example, if we need to export a large query result to a file then the paging will cause repeated requests to the server for more data. To make the export as efficient as possible, we can leverage the result streaming API introduced in RavenDB 2.5.

Here is an example of exporting the results of a query to an XML file:

```
using (var session = documentStore.OpenSession())
{
    var results = from page in session.Query<Page>() select page;
    using (var enumerator = session.Advanced.Stream(results))
    using (var writer = new XmlTextWriter("pages.xml",
    System.Text.UTF8Encoding.UTF8))
    {
        writer.WriteStartElement("pages");
        var serializer = new XmlSerializer(typeof(Page));
        while (enumerator.MoveNext())
        {
            var page = (Page)enumerator.Current.Document;
            serializer.Serialize(writer, page);
        }
        writer.WriteEndElement();
    }
}
```

Be aware that the results returned by the `Stream()` method are not tracked by session and, therefore, will not be automatically saved if modified. Also, the `Stream()` method uses RavenDB's export facility which snapshots the results when the export starts. If a document in the result is added or deleted while the results are streaming, this change will not be reflected in the results.

Being smart and lazy

There are cases when we need to retrieve multiple document results to perform a given operation. RavenDB optimizes this scenario by sending multiple requests in a single round trip to the server. To do this, RavenDB adds a `Lazily()` method to instruct the client to only issue the query when the data is actually accessed. When the data is needed, the requests are batched into a single multi-get request.

Let's say we have a page where we want to display the name of our Wiki space, followed by all of the pages tagged with **architecture**. This requires a load and a query. Take a look at the following code example to see how the `Lazily()` method is introduced to delay the server execution:

```
  var space = session.Advanced.Lazily.Load<Space>("space/1");
  var q = session.Query<Page>().Where(
  p => p.Tag.Contains("Architecture")).Take(10).Lazily();

session.Advanced.Eagerly.ExecuteAllPendingLazyOperations();

var author = space.Value.Title;
  foreach (var page in q.Value)
{

}
```

Lazily leverages the lazy loading class introduced with .NET 4.0. When the value on the space object is accessed, RavenDB jumps into action and sends the requests to the server. By calling the `ExecuteAllPendingLazyOperations()` method, RavenDB will send both of these requests to the server in a single round trip. This saves a round trip to the server plus RavenDB will process these queries in parallel as an added bonus!

Sometimes it is all about the little things

Doing large things fast is important, but doing little things quickly is just as critical. Since our documents are complex, there are times when we don't want to update the whole document. When making a small change to a document there is a good deal of overhead with loading the entire document from the server, modifying it on the client side, and sending it back to the server.

What we really need is the ability to send instructions to the server to make the change. For this, RavenDB provides a facility known as patching. The Patching API can add, update, or delete portions of a document.

While the patching feature can be useful for handling specific scenarios, it is not a general purpose tool. In many cases, you may be considering patching due to a poorly designed data model. Consider optimizing your data model before using the Patching API.

To see how this works, let's look at some examples. When a user simply renames the title of our Wiki page, we do not want to load and save the entire document. The `Patch()` command will make this small revision:

```
documentStore.DatabaseCommands.Patch(
    "pages/1",
    new[]
        {
            new PatchRequest
            {
                Type = PatchCommandType.Set,
                Name = "Title",
                Value = RavenJObject.FromObject("Design")
            }
        });
```

To handle concurrency, `PatchRequest` allows you to pass a `PrevVal` property which is compared against the current value to verify that a change will not overwrite new values. Since we are only modifying a portion of the document, this is usually preferable to using ETags. ETags are used for versioning the whole document and in this case we need our concurrency verification to be more granular. Here is the new `PatchRequest` request using `PrevVal`:

```
new PatchRequest
{
   Type = PatchCommandType.Set,
   Name = "Title",
   Value = RavenJObject.FromObject("Design"),
   PrevVal = RavenJObject.FromObject("Architecture")
}
```

Changing one document at a time is great, but what if we need to update a set of documents? RavenDB supports set-based operations for both updates and deletes. To perform a set-based operation, an index is utilized to select the documents to be included. Static indexes are recommended in this case to minimize the likelihood of stale data.

In our examples, we will use the following index.

```
public class PageIndex : AbstractIndexCreationTask<Page>
{
    public PageIndex()
    {
        Map = pages => from page in pages
```

```
                                select new {
                                    page.Title,
                                    page.Author,
                                    page.Tags
                                };

        }
    }
```

The interesting thing about this index is it has a nested collection **tags** that can be queried using this definition. Now searching for a page by tag is as easy as searching by title. Unlike a relational database, our index is not limited by a flat result set.

Let's also assume that we have added pages to our Wiki similar to the following:

```
session.Store(
new Page() { Author = "user/1",
Title = "Queue-based Processing",
Tags = { "Architecture", "Design" } },
"pages/1");
session.SaveChanges();
```

Using our index, we can search for a given tag and issue a patch request to move all of the documents into a given space in our Wiki simply by updating the space attribute.

```
documentStore.DatabaseCommands.UpdateByIndex("PageIndex",
        new IndexQuery { Query = "Tag:Architecture" },
        new[] {
                  new PatchRequest
                  {
                       Type = PatchCommandType.Set,
                       Name = "Space",
                       Value = RavenJObject.FromObject("space/1")
                  }
              }, allowStale: false);
```

We can also perform set-based deletes. The following example deletes all the pages authored by a specific user:

```
documentStore.DatabaseCommands.DeleteByIndex("PageIndex",
                                        new IndexQuery
                                        {
                                            Query = "Author:users/1"
                                        }, allowStale: false);
```


www.allitebooks.com

Patching meets scripting

There are some patching scenarios that are too complex for the simple methods we have discussed so far. RavenDB supports submitting a `ScriptedPatchRequest` request, which executes JavaScript server-side using Jint (a JavaScript interpreter for .NET). Be sure to test your syntax well since the script is passed as a string and does not benefit from compilation on the client.

Script patches can do almost anything. One handy use is to modify embedded collections within a document. For example, we could remove the design tag from our document with the following patch request:

```
documentStore.DatabaseCommands.Patch("pages/1",
    new ScriptedPatchRequest
    {
        Script = "this.Tags.Remove(tag)",
        Values = {{"tag", "Design"}}
    });
```

Note that the `Remove()` method is a special RavenDB function that is not part of normal JavaScript. The other thing to notice in this example is, you can pass variables into the script that saves you from nasty string concatenation.

Another powerful use case for script patches is maintaining denormalized data. Many times when designing our document data model, we will denormalize data to improve performance and simplify data access. This is especially true for elements that are read frequently, but updated infrequently. However, when the data does change we need a way to update the affected documents. Using the `LoadDocument` method in a set-based patch makes quick work of this problem.

The following example shows updating pages with the author's name when it changes on the user document:

```
documentStore.DatabaseCommands.UpdateByIndex("PageIndex",
    new IndexQuery { Query = "Author:user/1" },
        new ScriptedPatchRequest
        {
            Script = @"
                        var author = LoadDocument(this.Author);
                        this.AuthorName = author.Name;
                    "
        });
```

Using patching to migrate document schema

There are times when our schema design changes after documents have already been loaded into the database. Patching provides a performant way to migrate the documents into the new format. Simple migrations can be handled with the rename operations as follows:

```
documentStore.DatabaseCommands.UpdateByIndex("PageIndex",
        new IndexQuery(),
            new[] {new PatchRequest() {
                Type = PatchCommandType.Rename,
                Name = "Description",
                Value = "Title",
            }
});
```

More complex migrations can be handled by using a script patch. The following patch request parses a US formatted city, state, and zip address line and splits it into its individual components.

```
documentStore.DatabaseCommands.UpdateByIndex("UserIndex",
    new IndexQuery(),
        new ScriptedPatchRequest
        {
            Script = @"
                        var regex = new
                        RegExp('^(.+),\\s(.+)\\s(.*)?');
                        var match = regex.exec(this.Address2);
                        if (match!=null) {
                            this.City = match[1];
                            this.State = match[2];
                            this.Zip = match[3];
                        }
                    "
        });
```

Doing nothing is the fastest way to do something

We have discussed some fast ways to perform both large and small operations. But what is faster than this? Not exchanging data with the server at all! RavenDB has two layers of sophisticated client-side caching that will decrease the load on your database server and make your applications fly.

The first level of caching is the HTTP caching. Since the RavenDB server utilizes the HTTP protocol, it can leverage the caching built into the protocol. It does this by adding an ETag (or entity tag) to the header of every request/response. The ETag is a version identifier that RavenDB generates and also uses for optimistic concurrency. The first time `Load()` is called, the HTTP response provides the ETag header for the documents version. The client caches this response and the associated ETag.

When `Load()` is called again, the client sends the ETag from its cache up to the server. The HTTP GET request includes an HTTP If-None-Match header with the ETag's value. By checking the ETag from the client against the current value of the instance, the server can determine if it needs to send a fresh copy over the wire. If the ETags matches, the server simply returns a 304 Not Modified response to the client. RavenDB has a highly optimized code path to return the 304 response with very little cost. All of this is handled automatically by the API and server infrastructure. By default, RavenDB caches 2048 request by the following setting and it can be changed:

```
documentStore.MaxNumberOfCachedRequests = 2048;
```

This caching method is very efficient and saves bandwidth by not transferring documents multiple times from the client to the server. This is especially helpful when processing large documents. The only cost in this cause is the round trip from the client to the server to verify it has the latest version.

Getting aggressive about doing nothing

The second level of caching is known as aggressive caching. It was built to prevent the round trip from even happening. This is a zero-latency solution, but isn't as seamless as HTTP caching.

To use this caching, you will need to add a small bit of code around your calls:

```
using (session.Advanced.DocumentStore.
AggressivelyCacheFor(TimeSpan.FromMinutes(5)))
{
    var user = session.Load<User>("users/1");
}
```

The client library will cache results (including queries) for the time span specified. Prior to RavenDB 2.5, this feature had one downside: it was not aware of changes to the data on the server and could return stale data. While for some application scenarios this is not a problem, aggressive caching does need to be used carefully.

In RavenDB 2.5, this feature became even more powerful by automatically subscribing to changes from the server. The client uses these notifications to invalidate the cache and go back to the server for the updated results. This nearly eliminates the possibility of stale data, allowing you to cache the results for a longer period of time. However, you still need to be careful. If your client gets disconnected from the server, it may miss a notification and return stale data. The client will automatically re-subscribe, but may miss a notification while disconnected.

Summary

It is always important to use the right tool for the right job. The RavenDB API is no exception. Use the standard session-based APIs for 90 percent of your application and bring out these specialized APIs only when needed. However, when these APIs are used in the right place, they will make a huge difference in your application performance.

The foundation of our high performance application is now in place. We have explored building a great data model and how to tune the usage of the RavenDB API. In the next chapter, we will look at monitoring the application under load. This chapter will cover the tools available for locating bottlenecks and how to tune the server for optimal performance.

4
Monitoring and Tuning

Optimizing applications requires understanding how and when resources are being used. This insight will allow you to focus on solving the real problems and to avoid chasing rabbit trails.

In this chapter we will cover:

- Profiling web-based applications
- Debug logging
- Using performance counters
- Accessing server and database statistics
- Tuning your server for optimal performance

Profiling isn't just for crime solvers

In software development, profiling is the practice of gathering performance metrics while an application is running. This will help you determine if your application is efficiently using the RavenDB server. The RavenDB client comes with a built-in profiler that gives you insight into how your application is interacting with the server.

RavenDB provides access to the profiler with a web-based UI for ASP.NET MVC applications. By enabling the profiler, an orange button is included at the top of your page to view the RavenDB profile results during that web request. Each of your requests to the server will be listed with timing information.

To enable the RavenDB ASP.NET MVC profiler, do the following:

1. Add a reference to `Raven.Client.MvcIntegration`.
2. Add a line to your default layout (`_Layout.cshtml`) prior to the `</body>` tag. This will render the profiler UI:

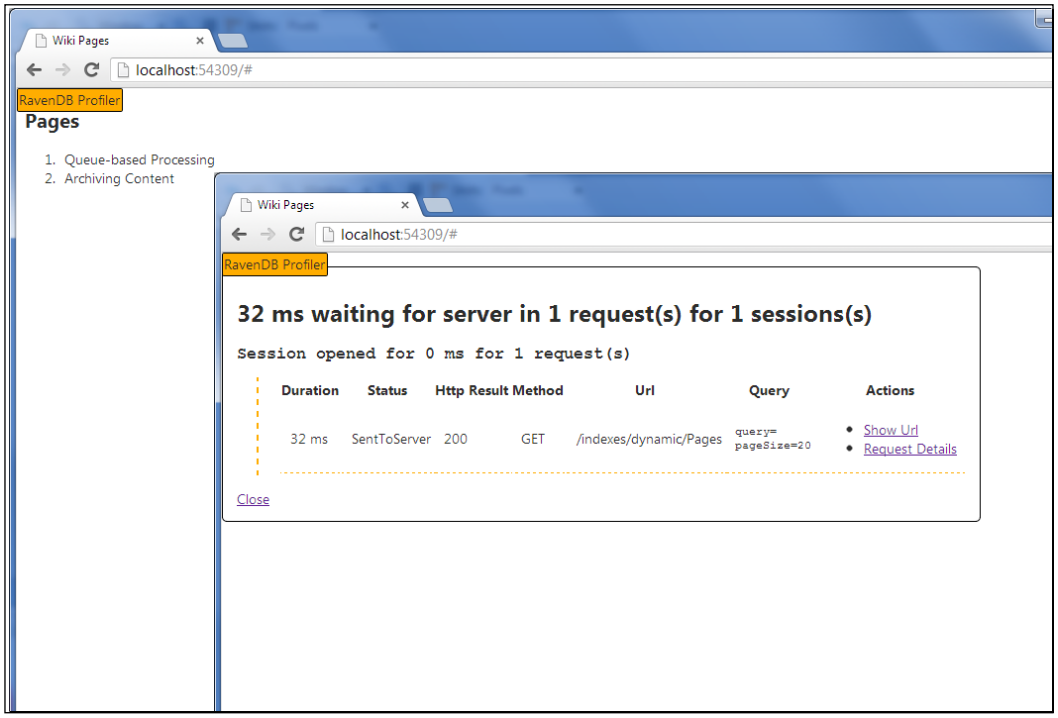
   ```
   @Raven.Client.MvcIntegration.RavenProfiler.
   CurrentRequestSessions()
   ```

3. Add a line to the `Application_Start()` method in `global.asax.cs`.

   ```
   RavenProfiler.InitializeFor(Store);
   ```

> Typically you do not want to show the profiler in production. However, it can be extremely useful for troubleshooting. You should either add code to enable/disable the profiler completely or just the user interface components.

Clicking on the **RavenDB Profiler** button shows a popup with detailed information about your RavenDB access as shown in the following screenshot:

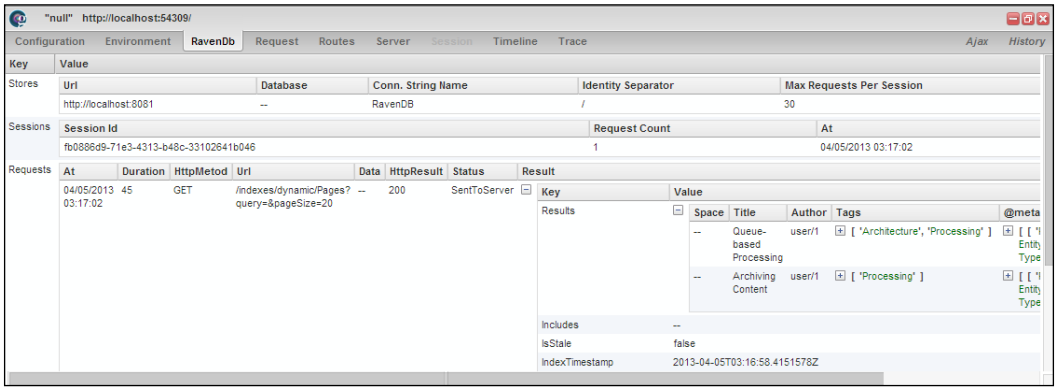


Clicking on the **Request Details** link will show the JSON that was returned from the server.

Profiling using Glimpse

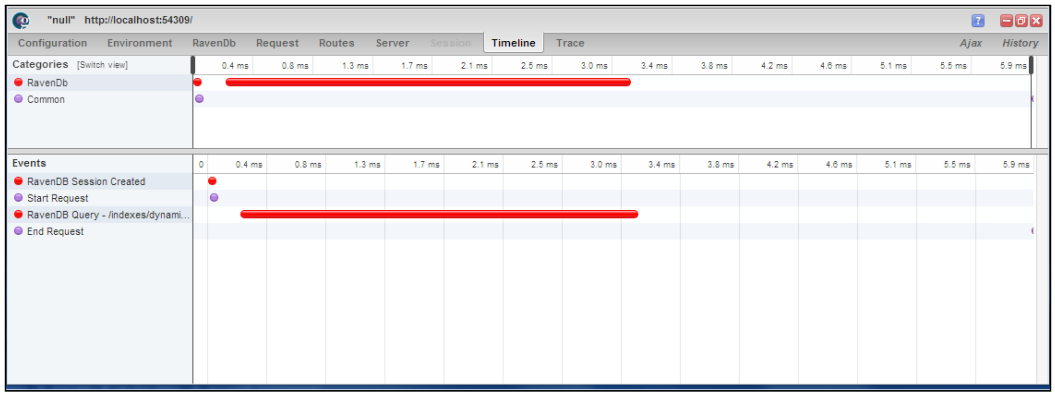
Another great option for viewing RavenDB profiler information is using Glimpse (`http://getglimpse.com/`). Glimpse is an open source diagnostics platform that provides a UI similar to **FireBug**, but instead of providing client-side information, Glimpse displays what occurred on the server.

Glimpse supports a plugin model for extending its core capabilities. Installing the RavenDB plugin provides information similar to the RavenDB profiler within the well-crafted Glimpse UI. The great thing about Glimpse is that it centralizes all of this information in one place, as can be seen in the following screenshot:



Once installed, the plugin adds a RavenDB tab which provides information about the document stores, sessions, and requests executed during this page request. It also allows you to explore the JSON returned in a nice grid-based interface.

The plugin also integrates the RavenDB requests into the timeline view giving you a visual representation of the request lifecycle as shown in the following screenshot:



To install Glimpse and the RavenDB plugin perform the following steps:

1. Install the packages using NuGet:

   ```
   Install-Package Glimpse.AspNet
   Install-Package Glimpse.RavenDb
   ```

2. Add lines to the `Application_Start()` method in `global.asax.cs`:

   ```
   RavenProfiler.InitializeFor(Store);
   Glimpse.RavenDb.Profiler.AttachTo(Store);
   ```

Securing Glimpse

Like the RavenProfiler, you will not want end user accessing Glimpse on your production site. Glimpse provides a facility to control access by implementing a class descended from `IRunTimePolicy`. When Glimpse is added to a project through NuGet, it creates a commented out class called `GlimpseSecurityPolicy`. Add your logic to the `Execute` method and return `RuntimePolicy.On` for users that you want to have access:

```
public RuntimePolicy Execute(IRuntimePolicyContext policyContext)
{
    if (!policyContext.GetHttpContext().User.IsInRole("SysAdmin"))
        return RuntimePolicy.Off;
    else
        return RuntimePolicy.On;
}
```

Monitoring with performance counters

RavenDB publishes aggregated performance information through **Windows Performance Counters**. These can be viewed using Windows Performance Monitor or integrated into your favorite enterprise monitoring tool for historical monitoring and alerting. MS System Center provides this enterprise monitoring, but there are a number of non-Microsoft solutions available as well. RavenDB also uses performance counter data from other subsystems such as **ESENT** and the **.NET CLR** to optimize its performance.

Creating performance counters requires administrator privileges and accessing performance counter data requires elevated privileges as well. However, like all applications, you should run RavenDB under a non-administrator account. RavenDB provides a command-line option to create the counters and place your user into the Performance Monitoring Group. To create the counters and give access to the user running the RavenDB server, execute the following command as an administrator:

```
Raven.Server.exe /user=USERNAME /setup-perf-counters
```

Upon successful completion, Raven will respond as follows:

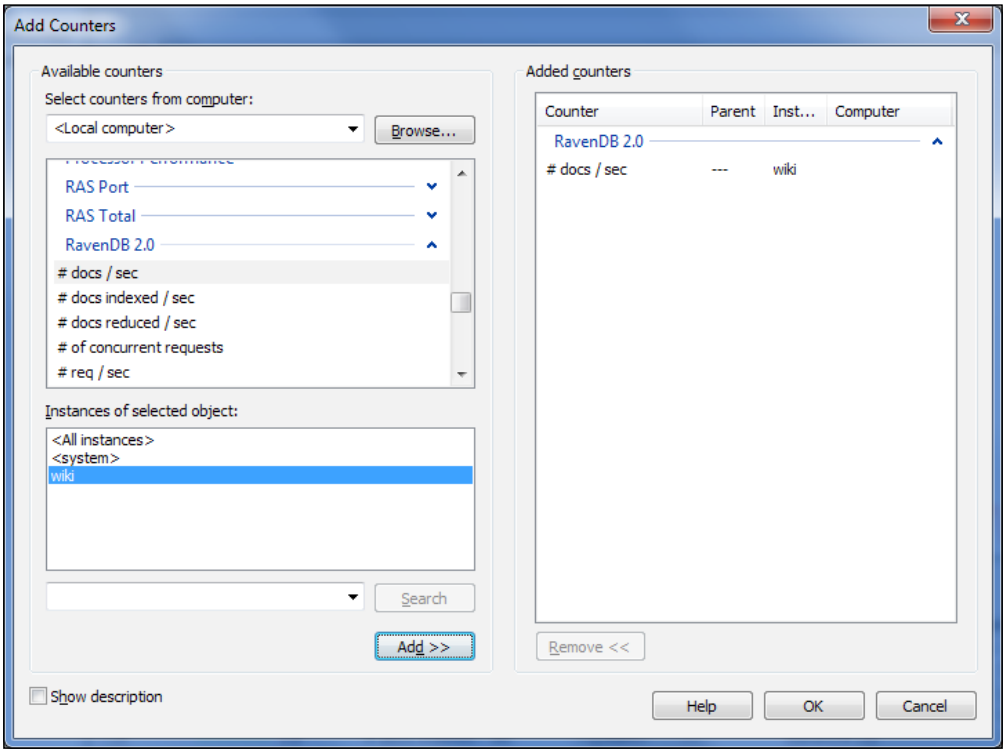
```
User USERNAME has been added to Performance Monitoring Users group.
Please log in the user again to take an effect.
```

While the counters are created, the user must log off to update the group membership. If RavenDB is running as a service or under an IIS app pool, you will need to restart the service or recycle the pool.

The following counters are now published:

Counter	Description
# docs/sec	number of document puts per second
# docs indexed/sec	number of documents indexed per second
# docs reduced/sec	number of documents reduced per second
# of concurrent requests	number of concurrent requests
# req/sec	number of requests per second

Now you can add the counters through Performance Monitor. Counters can be tracked for all databases (**<All instances>**) or a specific database (such as **wiki**) as seen in the following screenshot:



Looking inside using server and database statistics

RavenDB provides access to server information through a REST-based endpoint. Either through the browser or via code access: `http://SERVER:PORT/admin/stats` returns JSON providing uptime, memory usage, number of requests, and statistics on each database such as number of documents, requests per second, and more.

Database specific statistics are available at another REST-based endpoint: `http://SERVER:PORT/databases/DATABASENAME/stats`. The JSON returned includes information on your database such as configuration, indexing statistics, errors, and more. Many of these database statistics are also displayed within the RavenDB Studio. To access it through the Studio, click on **Statistics** under the **Status** menu.

Looking under the hood

If you need insight beyond what is provided by these tools, there is one more option. RavenDB uses NLog to write detailed information about what it is up to. NLog is an open source logging framework for .NET with multiple output targets ranging from simple files to queues and databases.

To enable NLog debug logging, simply place a file named `NLog.config` in RavenDB's base directory. The following sample code is provided by the RavenDB documentation, but the file can be customized to meet your needs. Reference the NLog documentation for more details: `https://github.com/nlog/nlog/wiki/Configuration-file`.

```
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.netfx35.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
   <targets>
      <target xsi:type="AsyncWrapper" name="AsyncLog">
         <target xsi:type="SplitGroup">
            <target name="File" xsi:type="File"
                   fileName="${basedir}\Logs\${shortdate}.log">
               <layout xsi:type="CsvLayout">
                  <column name="time" layout="${longdate}" />
                  <column name="logger" layout="${logger}"/>
                  <column name="level" layout="${level}"/>
                  <column name="message" layout="${message}" />
                  <column name="exception"
                  layout="${exception:format=tostring}" />
               </layout>
            </target>
         </target>
      </target>
   </targets>
   <rules>
      <logger name="Raven.*" writeTo="AsyncLog"/>
   </rules>
</nlog>
```

Tuning the server

Now that we have peered inside our database behavior at runtime, let's discuss what to do when things are not running optimally. First of all, realize that RavenDB is a self-tuning database server and usually does the best with resources at hand. However, there are some things you can do to improve performance in certain scenarios.

Since we are dealing with lots of data, disk I/O is usually the key to good performance. You should start by splitting the data, logs, and indexes onto different spindles. Typically, this is handled through a mechanism known as RAID. **Redundant Array of Inexpensive Drives** (**RAID**) is a set of data storage schemes that divide and/or replicate data among multiple hard drives for performance and/or safety. There are 10 RAID levels that are optimized for different use cases.

Logs are written sequentially and should be on **RAID1** configuration. Data and indexes are randomly accessed and would optimally be on a **RAID10** configuration. **RAID5** can be used in low write scenarios. Additionally, data and indexes will benefit greatly from a **solid state disk** (**SSD**) while the log can be left on a spinning disk. When using more advanced **storage area networks** (**SANs**), you will need to consult your storage vendor for the optimal configuration on their hardware.

To learn more about RAID levels, take a look at this RAID tutorial: `http://www.thegeekstuff.com/2010/08/raid-levels-tutorial/`.

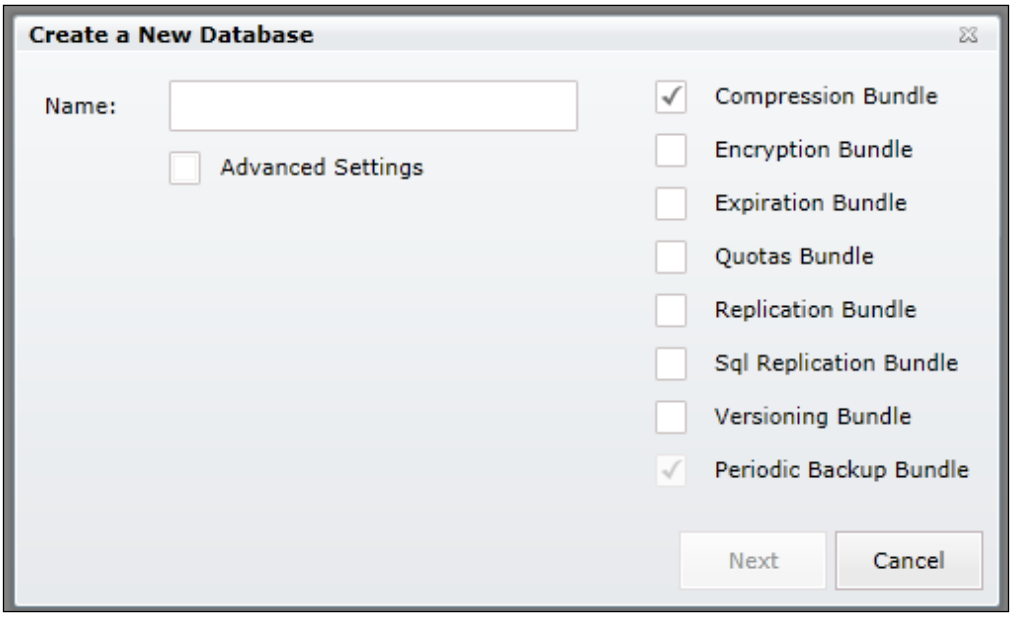
Less I/O = more speed

Another way to tune databases with high disk I/O is using the **Compression** bundle available with the Enterprise Edition of RavenDB. Compression will obviously make the database small on disk.

What might be less obvious is that the size of reads and writes go down as well. This is especially helpful for large documents. When the bundle is activated, RavenDB compresses all document writes and decompresses documents after they are read. There is additional CPU overhead, but if your application is I/O bound there is usually excess CPU capacity. An additional benefit of using the bundle is that backups are compressed as well.

Compression should only be enabled during database creation and not disabled once enabled. Data corruption will occur if this setting is changed once documents are stored in the database.

To enable compression, simply include the bundle during database creation through the RavenDB Studio as shown in the following screenshot:



Or use the API to include the bundle when creating the database through code:

```
store.DatabaseCommands.CreateDatabase(new DatabaseDocument
{
    Id = "CompressedDB",
    Settings =
            {
                // ...
                {"Raven/ActiveBundles", "Compression"}
            }
});
```

Tuning the settings

While RavenDB has lots of configuration settings, there are a few that you can easily tweak to optimize the database for your server. The following table explains how to take advantage of some of the key settings:

Configuration Settings	Description / Notes

`Esent/CacheSizeMax`
`MemoryCacheExpiration`
`MemoryCacheLimitMegabytes`
`MemoryCacheLimitPercentage`
	Configures the maximum size of the database page cache (ESENT) and internal RavenDB cache. Configure to larger value on systems with 8 GB or more of system memory.

`Esent/MaxVerPages`	The maximum size of version store (in memory modified data) available. The value is in megabytes. Impacts the number of concurrent transactions and the size of a transaction.
`Esent/DbExtensionSize`	The size the database file will be enlarged to when the file is full. Impacts how fast you can insert data. For frequent high insert rates, set the value higher.
`TransactionMode`	Safe transaction mode ensures data consistency but is slower. Lazy is faster but may result in a data loss if the server crashes.
`MaxNumberOfParallelIndexTasks`	Sets the number of indexing tasks that can be run in parallel. Reduce on slower systems.
`MaxNumberOfItemsToIndexInSingleBatch`	The maximum number of items that will be indexed in a single batch. Larger batch size results in faster indexing, but higher memory usage. Increase for large datasets when sufficient memory is available.

Summary

Understanding your server workload and configuration is the key to tuning for performance. Always be sure you are not creating extra load through poor programming practices. Then follow the tips described in this chapter to tweak out the most performance from your server. If you still need more speed, we will discuss scaling out across multiple nodes in the next chapter.

5
Scaling Out

As the load on your server increases, one server may not be sufficient to process the requests. While getting a bigger box may solve the problem in the short run, scaling up only works for so long and is an expensive option. Scaling out, on the other hand, is more cost effective and provides a path to extreme scalability.

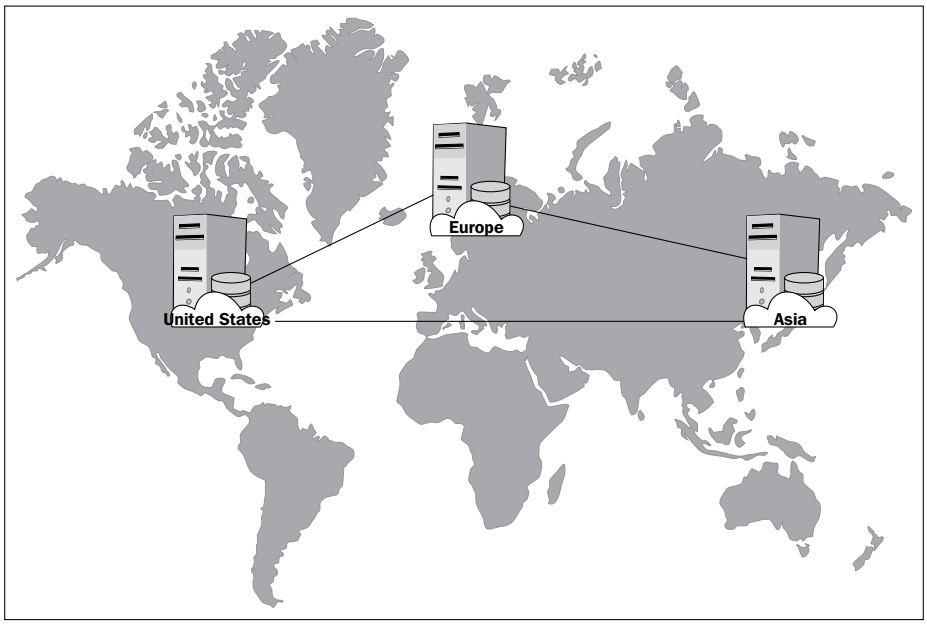
The Internet is built on the concept of scaling out. The Internet is not one big server. Even for a given site, requests can be load balanced amongst many servers in a server farm. The request/response nature of the HTTP protocol makes this very easy as long as things stay stateless. However, as soon as data or state is stored on a server, things get much more complicated. The user should get consistent results regardless of the server that is serving the request.

In this chapter, we will discuss how RavenDB solves these problems to scale your data across multiple servers.

Sharding

One solution to spreading data across multiple servers is known as **sharding** (also known as horizontal partitioning). Sharding splits the document database into shards (partitions), which can then be stored on disparate servers. Each server now becomes the owner for the documents in a given shard (or shards). Sharding is useful when you have large volumes of data or requests that overwhelm a single server. This works extremely well since each document is independent and can easily reside in separate databases. In a relational database, you need to worry about all of the related tables as well.

Sharding also works well for geographically dispersed data where the reads and writes are primarily within one graphic location. For example, the data in Europe is accessed 90 percent of the time by the team in Europe and 10 percent by teams outside Europe. With sharding, each region can have local access to its data and distributed access to the entire data set. The following diagram shows an example of a geographically distributed RavenDB cluster:

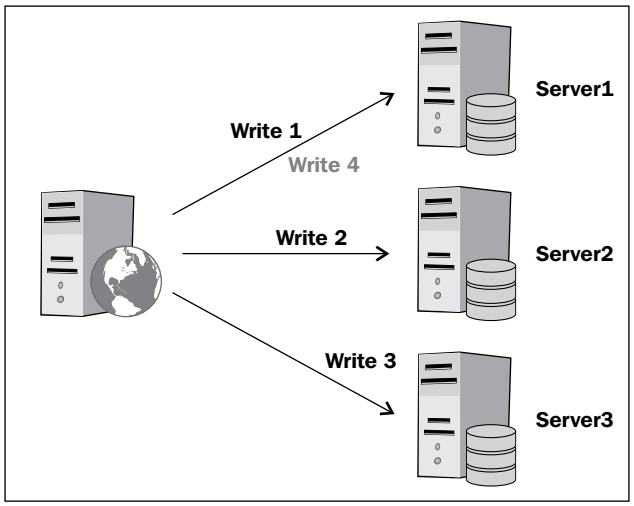


Unlike many other database systems, RavenDB configures sharding on the client, not on the server. The RavenDB client has built-in support for sharding documents. It supports multiple sharding schemes depending on the needs of the application.

Be aware that sharding does not work on attachments.

Sharding in the dark

The simplest form of sharding is known as **blind sharding**. With this option, RavenDB has zero insight into the structure of your data and simply load balances writes using round robin across the specified servers as shown in the following diagram:



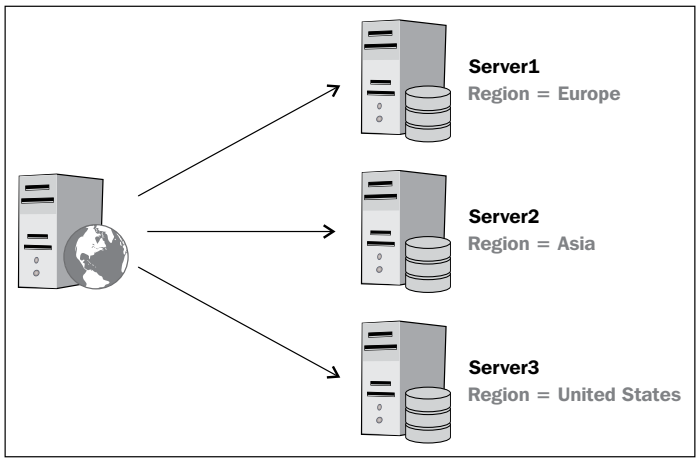
This is great for document writes, but since documents are effectively spread out randomly, it does not help with queries. Queries must be issued to all of the servers to determine which one has the documents in question. To utilize sharding, use shard `DocumentStore` and configure your `ShardStrategy`:

```
var shards = new Dictionary<string, IDocumentStore>
 {
  {"Shard1", new DocumentStore {Url = "http://server1:8080"}},
  {"Shard2", new DocumentStore {Url = "http://server2:8080"}},
  {"Shard3", new DocumentStore {Url = "http://server3:8080"}},
 };
var shardStrategy = new ShardStrategy(shards);
var docStore = new
ShardedDocumentStore(shardStrategy).Initialize();
```

The dictionary tells RavenDB the IDs of each shard and the corresponding server URL. When a document is stored using the `ShardedDocumentStore`, the client will prefix the document ID with the shard ID. So, a document stored with `ID customers/1` will be translated to `shard1/customers/1`. The shard ID is also stored in the document metadata as `Raven-Shard-Id`. This allows the client to automatically store the document into the correct shard when it is modified.

Sharding based on the data

To improve our sharding experience, we need to provide a bit more data intelligence to the RavenDB client. With a small change to our code, we can give RavenDB instructions on the boundaries on which to create shards. This is known as the partitioning key in the relational world. The sharding based on the data is shown in the following diagram:



Unlike in the previous example, writes are sent to a server based on data (in this case the region). This is accomplished by instructing RavenDB on what boundaries to shard our documents:

```
var shards = new Dictionary<string, IDocumentStore>
 {
  {"Europe", new DocumentStore {Url = "http://server1:8080"}},
  {"Asia", new DocumentStore {Url = "http://server2:8080"}},
  {"US", new DocumentStore {Url = "http://server3:8080"}},
 };
var shardStrategy = new ShardStrategy(shards);
shardStrategy.ShardingOn<DistributionCenter>(d => d.Region)
shardStrategy.ShardingOn<Shipment>(s => s.Region);

var docStore = new ShardedDocumentStore(shardStrategy);
docStore.Initialize();
```

Note that we have also located the `Shipment` document in the same shard as the `DistributionCenter` document to help optimize access in a geographically distributed cluster. It also allows us to utilize features such as includes and live projections since these functions require the documents to be located on the same server.

Now that RavenDB understands our data, it can also be more intelligent about where to send our queries. If the sharding key is used in the query, RavenDB will only send the request to the server that owns that shard:

```
var result = RavenSession.Query<DistributionCenter>()
                .Where(d => d.Region == "Asia")
                .OrderBy(d=>d.CenterName)
                .ToList();
```

In this case, only `server1` received the query and the other servers in the cluster do not receive any unneeded traffic. But what if we don't specify a shard key in our query?

```
var result = RavenSession.Query<DistributionCenter>()
                .OrderBy(d=>d.CenterName)
                .ToList();
```

In this case, it operates like blind sharding and broadcasts the query to all of the servers. The results are aggregated and sorted on the client.

Sharding based on – whatever you like!

So far we have looked at the simple ways to easily implement sharding on your database. Many times more control is needed and RavenDB provides the `IShardResolutionStrategy` interface for this purpose.

To better understand this interface, let's look at an example. Our system has a large number of one document type that needs to be load balanced across multiple servers. However, we do not want to take the overhead associated with query broadcasts for all of our other document types.

Let us suppose we are running a very large Wiki called Wikigalxi. In our Wiki, there are a huge number of pages, but much fewer other document types. The following resolution strategy puts all the documents except the `Page` document in a Common shard. The `Page` documents are then load balanced over a configurable number of shards:

```
public class ShardResolutionStrategy : IShardResolutionStrategy
{
    private readonly int numberOfShardsForPages;
    private int currentNewShardId;

    public ShardResolutionStrategy(int numberOfShardsForPages)
    {
```

```
            this.numberOfShardsForPages = numberOfShardsForPages;
        }

        public string GenerateShardIdFor(object entity,
                        ITransactionalDocumentSession sessionMetadata)
        {
            if (entity is Page)
            {
                var nextPostShardId = Interlocked.Increment(ref
                      currentNewShardId) % numberOfShardsForPages + 1;
                return "Pages" + nextPostShardId.ToString("D2");
            }
            else
                return "Common";
        }

        public string MetadataShardIdFor(object entity)
        {
            if (entity is Page)
                return "Pages01";
            else
                return "Common";
        }

        public IList<string> PotentialShardsFor(
                                  ShardRequestData requestData)
        {
            if (requestData.EntityType == typeof (Page) )
             return Enumerable.Range(0,
                numberOfShardsForPages).Select(i => "Pages" + (i +
                1).ToString("D2")).ToArray();
            else
                return new[] { "Common" };
        }
    }
```

Instead of using the `ShardOn()` method, we assign an instance of our class to the `ShardResolutionStrategy` property on the `ShardStrategy` object. In this case, I have broken my page documents into two shards `Pages01` and `Pages02`:

```
var shards = new Dictionary<string, IDocumentStore>
   {
     {"Common", new DocumentStore {Url = "http://server1:8080"}},
```

```
        {"Pages01", new DocumentStore {Url = "http://server2:8080"}},
        {"Pages02", new DocumentStore {Url = "http://server3:8080"}},
      };

foreach (var s in shards.Values)
{
    s.Initialize();
    s.DatabaseCommands.EnsureDatabaseExists("wiki");
}

var shardStrategy = new ShardStrategy(shards);
shardStrategy.ShardResolutionStrategy =
                    new PageShardResolutionStrategy(2);

var docStore = new ShardedDocumentStore(shardStrategy);
docStore.Initialize();
```

Notice, the `EnsureDatabaseExists()` method is being called against each `DocumentStore`. The reason for this is `ShardedDocumentStore` does not support this extension method and it must be called directly against each underlying database.

Loading a `space` document will automatically go against the `Common` shard:

```
var space = RavenSession.Load<Space>("Common/space/1");
```

While the document has the shard ID embedded in its ID, this is not what RavenDB is using to route the request. RavenDB uses the `IShardResolutionStrategy` interface to determine that this class belongs to the `Common` shard.

On the other hand, a query for a `Page` document will be broadcast to `server2` and `server3`.

```
var result = RavenSession.Query<Page>().ToList();
```

Custom sharding algorithms can be as complex or as simple as your application requires. The possibilities are truly endless.

Scaling out reporting to a relational database

RavenDB is a highly efficient database for storing documents and accessing them from your application. There are some cases, such as reporting, where a relational database provides better capabilities and tooling. This also provides another scale out point to move the reporting load onto a separate infrastructure. One example would be to replicate your reporting data into a Microsoft SQL Server database and provide reporting through Microsoft SQL Server Reporting Services.

To set up replication to a relational database, you should use the included **SQL Replication** bundle. This bundle replaces the previously supported Index Replication bundle. The new bundle is easier to use and does not suffer from dropping documents if a connection failure occurs.

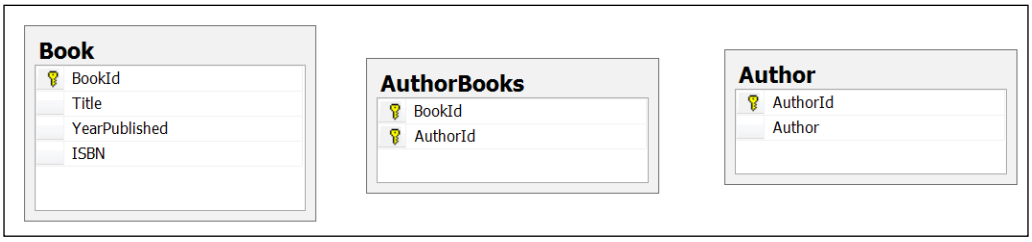
Due to the complex nature of documents stored in RavenDB, it is necessary to instruct RavenDB how to map the documents into the relational structure. Enabling SQL replication can be accomplished using the UI by adding new replication settings by going to **Database | Settings | SQL Replication** or through code by saving a configuration document under `Raven/SQLReplication/Configuration/ReplicationName`.

To understand how this works, lets walk through an example. Using the example from *Chapter 2*, *Optimizing the Data Model*, our database contains the following documents:

```
// books/1
{
   "title": "CLR via C#",
   "ISBN": "978-0735667457",
   "published": 2012
}

// books/2
{
   "title": "Applied Microsoft .NET Framework Programming",
   "ISBN": "978-0735614222",
   "published": 2002
}

// authors/1
{
  "name": "Jeffrey Richter",
```

```
    "books": [
        { "id":"books/1", "title":" CLR via C#"},
        { "id":"books/2",
          "title":" Applied Microsoft .NET Framework Programming"} ]
  }

  // authors/2
  {
    "name": "Maarten van de Bospoort",
    "books": [ {"id":"books/1", "title":" CLR via C#"} ]
  }
```

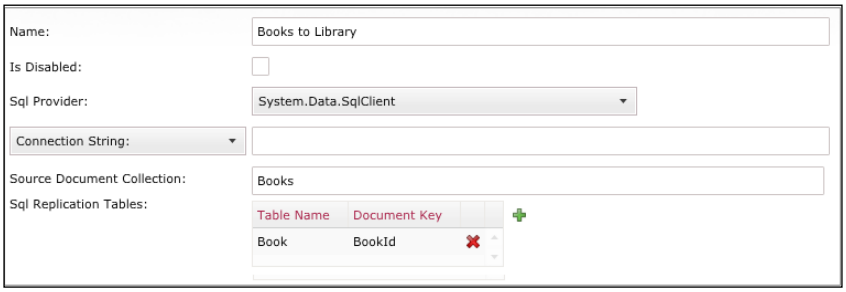
There are two collections of documents: books and authors. But, the relational database is a bit more complex since SQL needs an additional entity to represent the many-to-many relationship. The relational tables for our reporting look like the following diagram:



Since the documents may not be replicated in a specific order, the tables do not have foreign key constraints specified. With foreign key constraints, if an `Author` document was replicated before a `Book` document, the insert would fail.

Now that the structures are in place, it is time to configure replication. Each document collection must be configured independently. On the **SQL Replication** tab, click on the **Add New** button to set up the first replication as shown in the following screenshot:

Since the `Book` document is the simplest, we will start there. After specifying a descriptive name, specify the **SQL Provider** and **Connection String** of the replication target as shown in the following screenshot. For SQL providers, RavenDB supports SQL Server, SQL CE, Oracle, MySQL, PostgreSQL, and OleDb. The following screenshot shows the replication configuration:

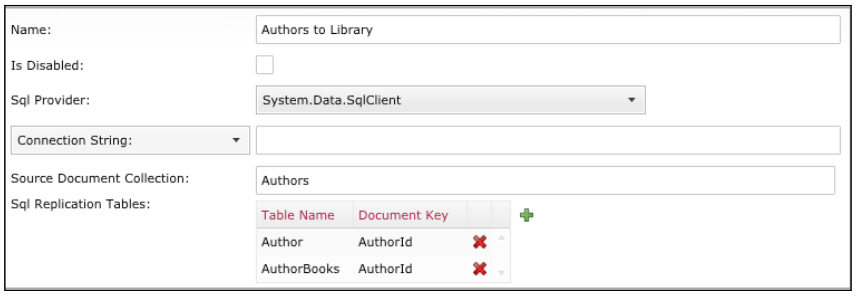


The **Source Document Collection:** is based on the document metadata property called `Raven-Entity-Name` which is added automatically by the RavenDB client. The **SQL Replication Tables:** area is a list of target tables that will be populated by the replication process. In this case, we are replicating to a table named **Book** and placing the document key into the field called **BookId**. Other field mappings are handled through a snippet of JavaScript.

The following script handles mapping the book fields. The keyword represents the current document. A special function `replicateToBook()` is used to send data to the target table. RavenDB will make a function available for each table specified in the **SQL Replication Tables:** list:

```
var book = {
   Title: this.title,
   YearPublished: this.published,
   ISBN: this.ISBN
};

replicateToBook(book);
```

Next, let's tackle the `Authors` document collection. It is more complex if there is a nested collection of book references. Adding the replication setting starts in the same way, but has an additional table specified in the **SQL Replication Tables:** as shown in the following screenshot:



Note that the **AuthorBooks** table maps the **Document Key** exactly in the same manner as the **Author** table. In this case, we are not specifying the complete key for the relational table, just which column maps to the **Document Key**. The script handles the complex details.

The following script first replicates the **Author** and then loops through the book references and replicates them to the **AuthorBooks** table.

```
var author = {
  Author: this.name
};

replicateToAuthor(author);

for (var i=0;i<this.books.length;i++)
{
  var authorBook = this.books[i];
  var bookReference = {
    BookId: authorBook.id
  };
  replicateToAuthorBooks(bookReference);
}
```

Click on the **Save Changes** button. It will create the configuration documents and trigger the replication to start. Now the following SQL query can be written against the tables:

```
select b.BookId,b.Title, b.YearPublished, b.ISBN,
a.AuthorId, a.Author
from book b
inner join authorbooks ab on b.BookId=ab.BookId
inner join author a on ab.AuthorId = a.AuthorId
```

And produces the expected results as shown in the following screenshot:

Results | Messages

	BookId	Title	YearPublished	ISBN	AuthorId	Author
1	books/1	CLR via C#	2012	978-0735667457	authors/1	Jeffrey Richter
2	books/1	CLR via C#	2012	978-0735667457	authors/2	Maarten van de Bospoort
3	books/2	Applied Microsoft .NET Framework Programming	2002	978-0735614222	authors/1	Jeffrey Richter

If RavenDB has problems replicating documents or connecting to the SQL server, errors will show up on the status page in the logs or alerts tabs.

Summary

The scale out technology in RavenDB gives your application the ability to scale beyond the confines of a single large server. Take care to design your sharding strategy to balance complexity and performance. If at all possible, stay away from blind sharding. Along with sharding, replicating data to a relational database allows the reporting aspects of your application to access a separate storage infrastructure and takes advantage of existing reporting tooling. In the next chapter, we will discuss replication for fault tolerance and see how it can be used as another scale out option.

6

When Failure is not an Option

As you scale up your users, the RavenDB server will be a critical piece of your infrastructure. Planning for failure is a key step for providing high availability and uptime to your users.

For high availability, RavenDB provides the following:

Type	Shared resources	How it works
Replication	Shared nothing	Documents are shipped from the source server to one or more destination servers on add, update, or delete.
Failover Clustering	Shared disk	Each node accesses shared database storage, but only one node is active at a time.

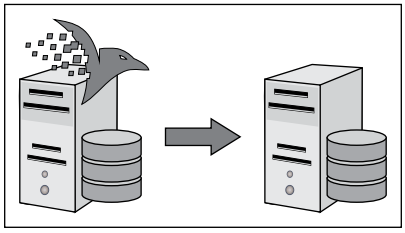
Replication

RavenDB provides built-in facilities to enable simple yet powerful replication mechanisms that can be configured in various topologies. Be aware that RavenDB only replicates documents and attachments. It is up to you to create the appropriate indexes on each instance. This is by design to get you maximum flexibility for optimizing each instance for its use. If you always want the same indexes on the replicas, simply create the index using the client API. This will automatically create the indexes on the primary and replicas.

In the diagrams of this chapter, the RavenDB logo denotes a node is running an active instance of RavenDB. If the node does not have the logo, it is still running the server but is not actively in use by clients.

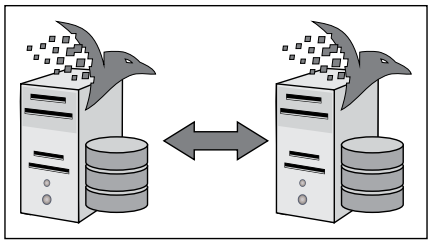
Master-slave

In a master-slave arrangement, the master server is used for both reads and writes. The slaves receive a copy of the changes, and the copy is available if the master fails. The following diagram shows two servers in this configuration:



Master-master

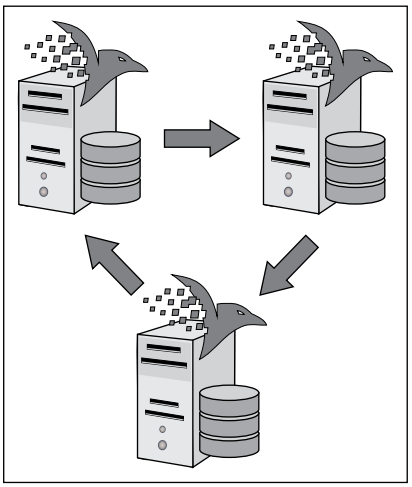
In a master-master topology, both servers are being written to and read from. This spreads the load across the nodes which can improve scalability. However, it is now possible for conflicts to occur when the same document is changed on each server. The following diagram shows reads and writes going to both servers:



It is important to create some form of logical isolation to decrease the chance of conflicts.

Multi-master

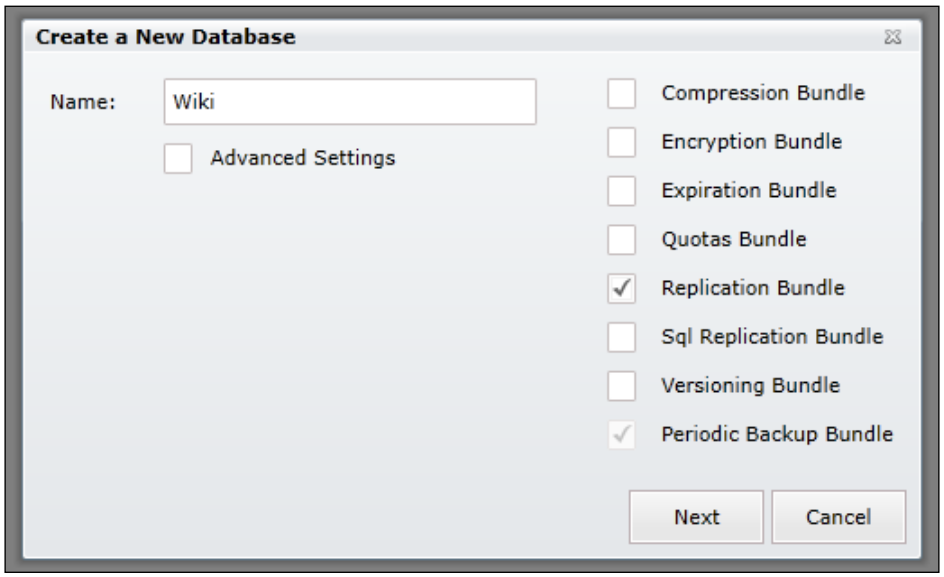
This topology is typically used to support geographically separated nodes, where all the data is needed and accessed at each location. Each node replicates to one of the other nodes in the cluster. Database clients at each location write to their assigned server and the changes are replicated throughout the cluster. This topology is rarely used in practice due to conflicts and lack of client API support.



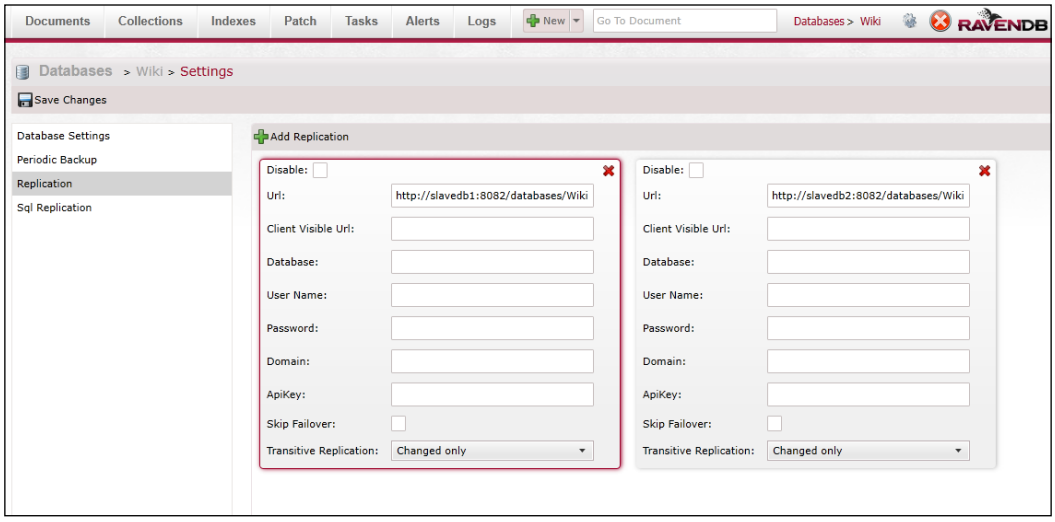
Configuring replication

Replication is configured through the source RavenDB instance. Regardless of the topology, replication is configured at the source instance. Once the source instance is configured, replication will automatically send changes out to the destination.

To configure your database for replication, you must enable the **Replication Bundle** during database creation on both the source and destination. This can be accomplished through code or using the UI as follows:

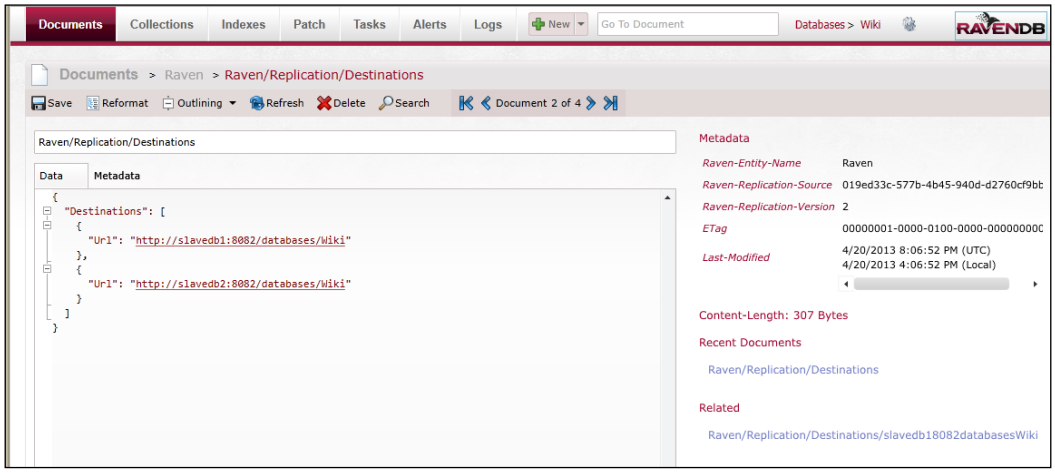


Next, configure the source instances to replicate documents. Through RavenDB studio, click on the database settings and add one or more replication destinations as follows:

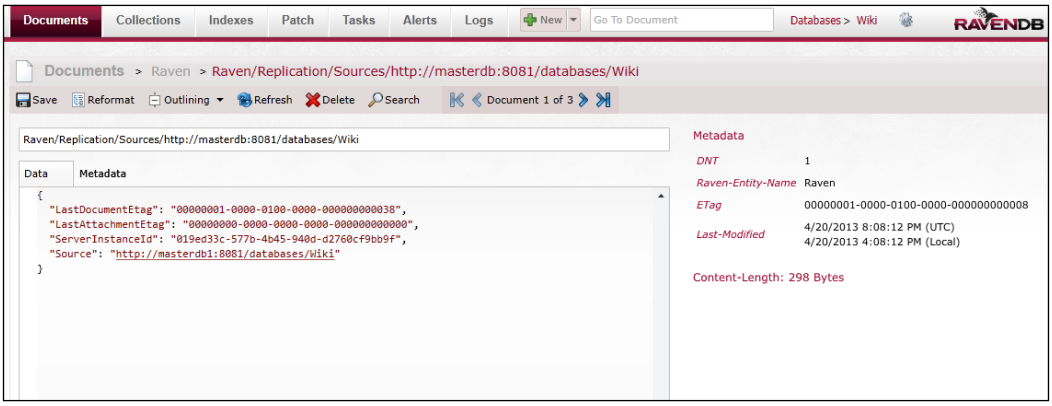


RavenDB supports multiple replication destinations. This allows for improved fault tolerance or replicating a copy for another purpose such as read stripping which is discussed later in this chapter.

When you save these settings, RavenDB creates a system document that stores the destinations. This document can be read or updated directly if you need programmatic configuration. The following screen shows the system document:



Once replication starts, the destination server will also create a new system document describing the source of the replication.



Handling conflicts

Whenever you have more than one write-enabled instance, conflicts can occur. Conflicts are typically handled on the client side, which allows the user to decide which version to keep. The following sample controller has a `catch` block to handle the `ConflictException`:

```
public ActionResult Index(string id)
{
    try
    {
        var page = RavenSession.Load<Page>(id);
        return View(page);
    }
    catch (ConflictException e)
    {
        var list = new List<JsonDocument>();
        foreach (var v in e.ConflictedVersionIds)
        {
            var doc = documentStore.DatabaseCommands.Get(v);
            list.Add(doc);
        }
        return View("ResolvePageConflict", list);
    }
}
```

In this case, a `ConflictException` is raised and it includes the identifiers for each copy of the document in conflict. A new `View` is sent down to the user to select which version to keep. Once the user has selected the correct version, the controller that receives this page will update the document in the database:

```
public ActionResult ResolvePageConflict(string id,
  string resolvedData, string resolvedMetaData)
{
    documentStore.DatabaseCommands.Put(id, null, resolvedData,
    resolvedMetaData);
    var page = RavenSession.Load<Page>(id);
    return RedirectToAction("Index");
}
```

The `Put` will overwrite the conflict with the version selected by the users.

Automatically handling conflicts

Another option for handling conflicts is to take the manual action out of the user's hands. It is possible to teach RavenDB how to handle conflicts by implementing an `AbstractDocumentReplicationConflictResolver`.

In many cases, just using the latest version of a document is the correct choice. However, it is important to think through the ramifications of this choice. John Bennett has developed a `LastInWinsReplicationConflictResolver` during the implementation of RavenDB for MSNBC. The resolver compares the `LastModified` value in the metadata of each document. The one with the most recent value automatically wins.

The code is deployed on the RavenDB server as a bundle, which extends the built-in functionality. You can download the code from his blog at `http://jtbennett.com/blog/2012/03/customizing-ravendb-a-simple-ravendb-server-bundle-for-replication-conflict-handling`.

A careful analysis of your business needs and data changes is required to make an informed decision.

Handling node failures

When a node in the replication cluster fails, the RavenDB client will automatically begin accessing data from the replicas. This is the default behavior, but there are other options.

To configure the failover behavior, use the following code:

```
documentStore.Conventions.FailoverBehavior =
  FailoverBehavior.FailImmediately
```

The following options are available:

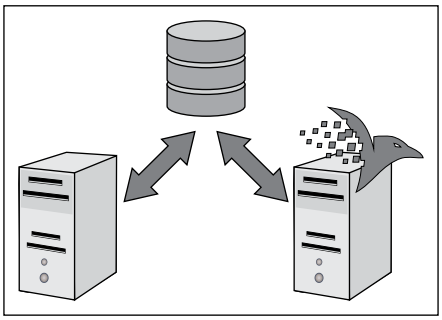
Failover Behavior option	Description
`AllowReadsFromSecondaries`	Default option. Reads from replica servers on a failure of the primary node. Writes are disabled.
`FailImmediately`	Turns off failover.
`AllowReadsFromSecondariesAndWritesToSecondaries`	Reads & writes to replicas on primary failure. Set up master-master replication to ensure changes, primary node receives changes when it comes back up.
`ReadFromAllServers`	Enables read stripping. Discussed in more detail later in this chapter.

Once the primary node comes back up, the client will automatically shift access back to the primary node.

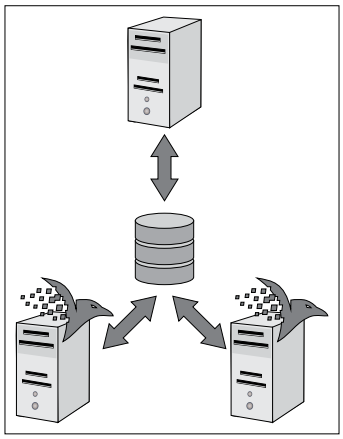
Failover clustering

The Enterprise Edition of RavenDB supports Failover clustering as another mechanism for providing fault tolerance to your application. Failover clustering supports a multi-node cluster where only one node is active for the instance at any given point in time. When the active node fails, the cluster will automatically migrate the instance to one of the other nodes.

The following diagram shows a shared disk attached to each of the servers:



It is possible to set up an active-active cluster by hosting multiple instances on separate nodes. Optimally, you should have *N+1* nodes so there are sufficient resources if one node fails. For example, if you could have a three node cluster with two instances of RavenDB running. Alternatively, each server should only be loaded with 50 percent so that it can handle running its own instance plus the instance from a failed node. The following diagram shows a multi-instance configuration:



Failover Clustering relies on the following:

- Windows Server 2008 or greater with Failover Cluster Manager snap-in installed
- A shared disk provided by a **Storage Area Network** (**SAN**) or other mechanism
- RavenDB installed as a service (not hosted in IIS) on every cluster node and stopped

- RavenDB configured with its data directory residing on the same location on the shared disk (set `Raven/DataDir` in `Raven.Server.exe.config`)

Once you have the prerequisites installed, you are ready to configure clustering through the Windows Failover Cluster Manager. The first step is to create the cluster. This involves informing Windows the names of the nodes in your cluster, specifying a cluster name, and providing a static IP for the cluster.

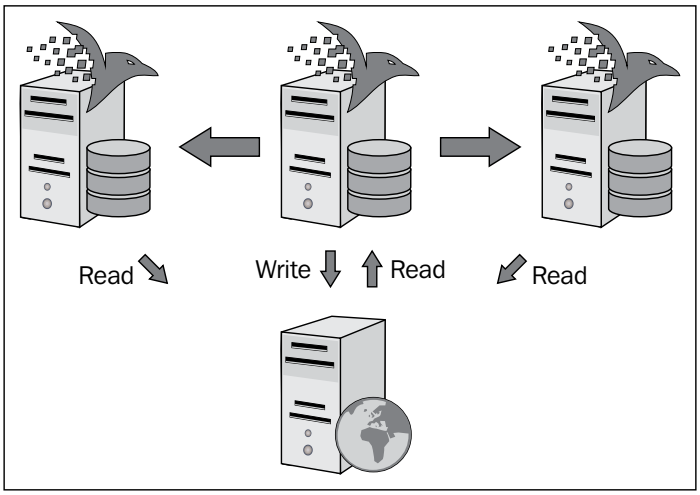
Next, tell the cluster to provide high availability of the RavenDB service. You will need to specify a name for the clustered service and provide a static IP address for connections. Your client application will use this address to connect to the cluster, regardless of which node is actively hosting the instance.

Once the service is configured, you can also tell Windows if there is a preferred node to run a given instance of the service. This can be useful if not all of the nodes have the same resources. For cost reasons, you may have a less powerful server as the backup node.

> For a step-by-step guide with screenshots, see the RavenDB documentation at `http://ravendb.net/docs/2.0/server/administration/fmc_configuration`.

Read stripping

Read stripping is a process that accelerates reads and improves fault tolerance by load balancing them across multiple servers. Sending writes to only one server in the cluster prevents replication conflicts. The following diagram shows this configuration:



To use read stripping perform the following steps:

1. Connect to the write master:

   ```
   var docstore = new DocumentStore() { Url =
     "http://masterdb:8081" };
   ```

2. Specify `ReadFromAllServers` behavior. This will load balance session reads across the servers in the cluster. It automatically configures the endpoints by loading the replication destinations document:

   ```
   docstore.Conventions.FailoverBehavior =
     FailoverBehavior.ReadFromAllServers;
   ```

3. Initialize the document store:

   ```
   docstore.Initialize();
   ```

4. Calls to `documentstore.OpenSession()` will now perform round robin between instances. Multiple queries on the same session are sent to the same server. Writes will only be sent to the write master.

Within the current session, all reads access the same server ensuring a certain level of read consistency for that session. However, once you close the session, all bets are off. The reason for this is that each of the replicas can be at a different state if the cluster is receiving writes. Therefore, read stripping is best for either read-only databases or databases that are updated nightly when most users are off the system.

Another use case for read stripping is a database that only receives updates from external sources, not from the user directly. Since the user does not have visibility into the pending writes, their view will always look current regardless of which replica they access. However, if they access different replicas within the same session this can lead to strange results. One way to solve this problem for web applications is to load balance based on user session instead of database session. This is similar to HTTP load balancers assigning a "sticky session". To do this with RavenDB, we need to override the `ReplicationInformer` class as follows:

```
public class UserSessionAlignedReplicationInformer :
  ReplicationInformer
{
    static int NextSessionID;
    static object LockHandle = new object();

    public UserSessionAlignedReplicationInformer
    (DocumentConvention conventions) : base(conventions)
    {
    }
```

```
    public override int GetReadStripingBase()
    {
        int? SessionID = HttpContext.Current.Session
["RavenDB-Replication-SessionID"] as int?;
        if (SessionID==null)
        {
        Interlocked.Increment(ref NextSessionID);

            HttpContext.Current.Session
["RavenDB-Replication-SessionID"] = SessionID;
        }
        return (int)SessionID % ReplicationDestinations.Count + 1;
    }
}
```

This class will assign a `SessionID` and store it in the user's session. The session is then used to calculate the number of servers to send requests. This ensures that a user does not get assigned to a new server on each database session.

To use a custom `ReplicationInformer`, implement `ReplicationInformerFactory` as in the following code:

```
var docstore = new DocumentStore() {
Url = "http://masterdb:8081"
};
docstore.Conventions.FailoverBehavior =
  FailoverBehavior.ReadFromAllServers;
docstore.Conventions.ReplicationInformerFactory = (database) =>
{
    return new UserSessionAlignedReplicationInformer(
    new DocumentConvention());
};
docstore.Initialize();
```

By aligning the database session to the user session, the reads for a given user will hit the same server and give them a consistent view of the data.

Summary

RavenDB provides enterprise grade options to make your data highly available. Picking between replication and clustering should be based on the business needs of your application. Replication can also be used to improve scalability by offloading access to different servers. However, it is important to consider conflicts and consistency in the design of your architecture. In the next chapter, we will dive into extending the server-side capabilities of RavenDB.

7
Deploying to the Cloud

The cloud holds a great deal of promise for the future of computing, but there is also risk in moving towards this model.

When moving to the cloud, you should consider the following:

- Will you use the cloud as your primary instance or a backup instance?
- What is the capacity of the cloud vendor? What is the plan if you outgrow their capacity?
- What are the up-time requirements of your application?
- What is the **recovery point object** (**RPO**) and **recovery time object** (**RTO**) in the case of a failure? In other words, how much data can you lose and how long can you be down?
- Does the cloud vendor's security model match the one required for your application?
- How does the vendor's pricing model affect your business as the application scales? Does your cloud provider require you to run multiple instances? Can they guarantee a single instance SLA?

Another point to consider is which of the cloud computing options for RavenDB best fits your application's requirements. There are basically three different ways to run RavenDB in the cloud, which are stated as follows:

- Platform as a Service
- Infrastructure as a Service
- Database as a Service

www.allitebooks.com

Platform as a Service (PaaS)

Platform as a service vendors like Microsoft Azure provide a managed .NET platform for deploying websites and worker processes. Since RavenDB is a .NET application, it may seem like a good idea to deploy it as a worker process.

There are a couple of folks who have built solutions to perform just that:

- `https://github.com/SaschaDittmann/RavenDbOnAzure`
- `https://github.com/jalchr/RavenDb.Azure`

However, the downside of Azure managing your platform is that it will randomly restart your processes. For stateless processes, this is not a big deal. For a database, this is a big problem.

> Until there is a good solution for this problem, Azure web processes or worker processes are not recommended.

Infrastructure as a Service (IaaS)

Since Platform as a Service is a no go, what about Infrastructure as a Service (IaaS)? IaaS provides a raw virtual machine that can be configured as needed. Examples of this are Amazon EC2 or Microsoft Azure Infrastructure Services.

Since you have full control, the random restarts are no longer a concern. However, since you have a raw VM, all of the support is left to the developer or operations team. This works well if you have a good system administration team behind you.

Database as a Service (DaaS)

When deploying RavenDB to the cloud, Database as a Service offers a number of compelling advantages, a few are stated as follows:

- No infrastructure administration: DaaS companies handle the provisioning, upgrading, and maintenance of the entire software stack: from operating system through the RavenDB server.
- Optimized for RavenDB: Unlike IasS, the RavenDB instance will not run on a generic virtual machine (VM). The vendors know the product and can optimize it for your use.

- Fixed monthly cost: Known cost model that includes processing cycles and I/O.

Today, there are two primary vendors providing this service: RavenHQ and CloudBird. They have very similar capabilities:

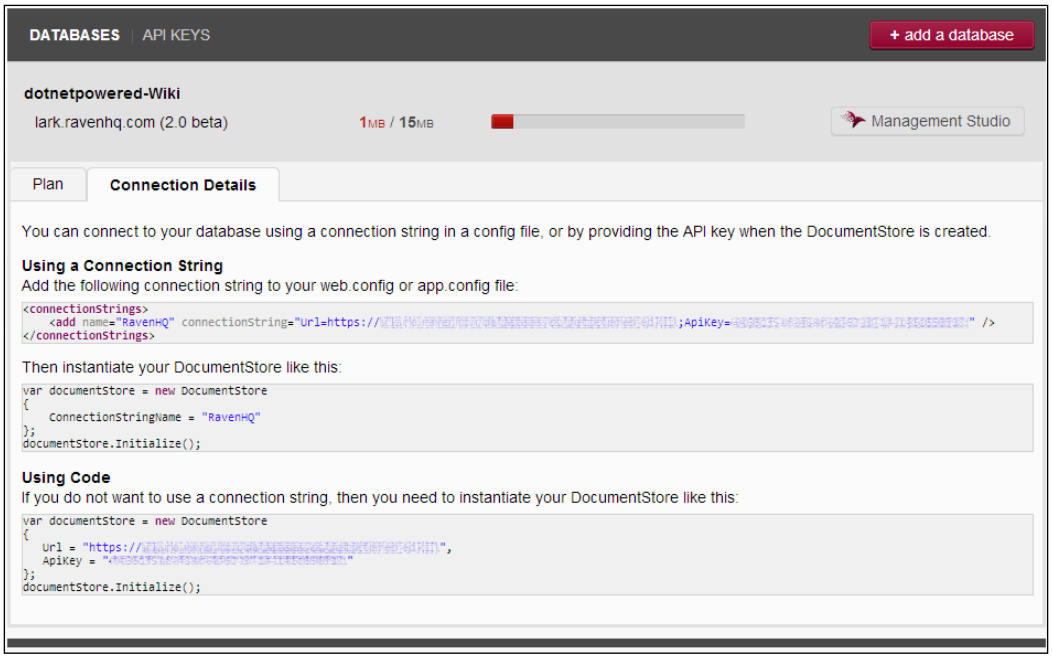
	Location	Versions	Backups	Integration	Replication
RavenHQ	US East & EU	1.0 or 2.0	Daily, hourly to Amazon S3	AppHarbor	Available upgrade
CloudBird	EU (US coming soon)	1.0 or 2.0	Daily, hourly to Amazon S3	AppHarbor	Available upgrade

Getting started is easy

Both services provide a free option to get started. When you need more capacity, performance, or replication it is a simple upgrade. Creating a database is as simple as naming it and waiting for less than 60 seconds for the provisioning process to complete. Once your database is created, a page is displayed with the connection information and the space usage. You also have easy access to launch the Raven Management Studio.

CloudBird presents the following screen after the database is created, providing connection information:

RavenHQ provides a similar screen with connection information and one click access to the RavenDB Management Studio:



The best way to connect to a cloud store is by using a connection string with an embedded API key. This allows you to connect to the cloud or your local instance with the same code:

```
<connectionStrings>
  <add name="Raven" connectionString=
  "Url=https://cloudserver/databases/wiki;ApiKey=x" />
</connectionStrings>

var documentStore = new DocumentStore
{
    ConnectionStringName = "Raven"
};
documentStore.Initialize();
```

Alternatively, you can specify the URL and API key separately. But this will most likely require code changes unless you are always using a cloud data store.

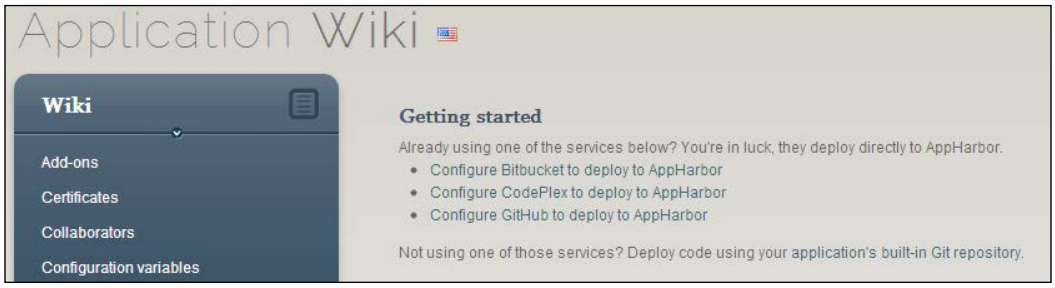
```
var documentStore = new DocumentStore
{
```

```
        Url = "https://cloudserver/databases/wiki",
        ApiKey = "xxxxx-xxxx-xx-xxx-xxxxxxx"
    };
    documentStore.Initialize();
```

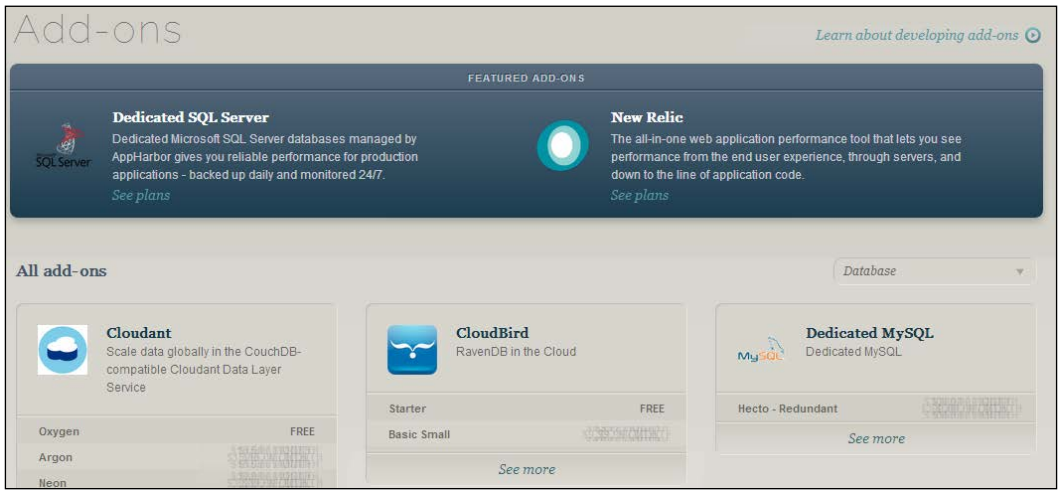
AppHarbor Integration

AppHarbor is a PasS vendor providing simplified hosting of .NET-based web applications. Simply connect your BitBucket, CodePlex, or GitHub code repository and it will automatically build and deploy your application. Both RavenHQ and CloudBird are integrated add-ons that give you centralized billing and management through the AppHarbor interface.

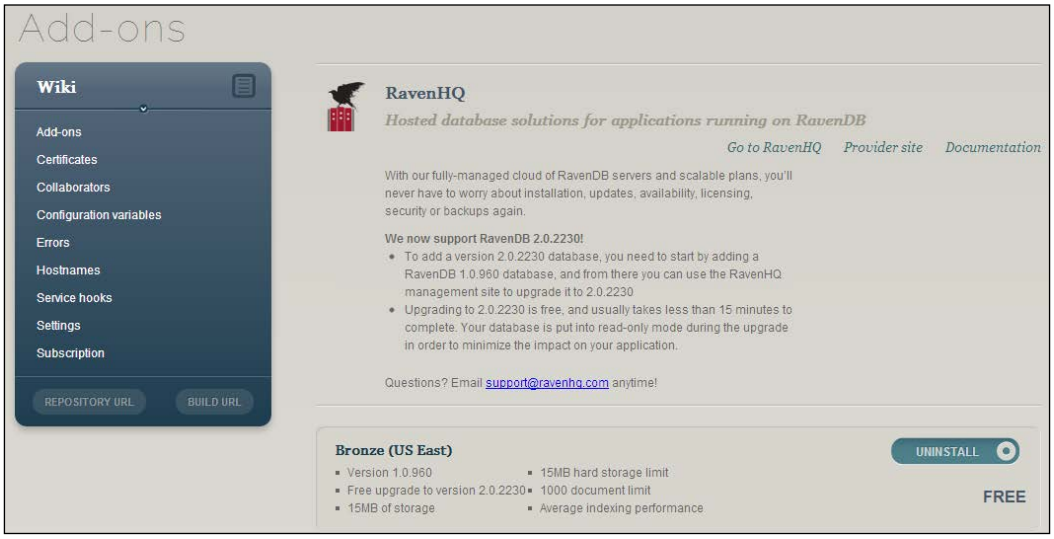
Once you have registered for AppHarbor and created your first application, click on the **Add-ons** option as shown in the following screenshot:



Both RavenHQ and CloudBird will appear in a long list of different third-party services. Click on the **See more** button to read more and install one of the packages:



After the installation completes, you will see the following screen (RavenHQ in this example). Click on the **Go to RavenHQ** link to get the connection information and manage the database:



Summary

Deploying your application along with a document database like RavenDB to the cloud has many advantages. However, it is important to evaluate your application requirements against the capabilities of your cloud vendor. Once you have decided to move to the cloud, select a DaaS vendor that best meets your needs. The exception to this rule is when the application requires custom extensions to the database server. In the next chapter, we will explore the server extension options.

8
Extending RavenDB

RavenDB provides a number of extension points for adding functionality directly to the server. Since RavenDB is written in .NET, it is trivial for .NET developers to take advantage of these features. In this chapter, we will discuss a few of them: triggers, codecs, and tasks. Each of these are extensions and are used by implementing an interface or by descending from an abstract class. Once an extension is built into a separate assembly (DLL), it is placed in the plugins directory. On startup, Raven DB uses **Microsoft Extensibility Framework** (**MEF**) to automatically discover and wire-up your extensions.

Triggers

Triggers are an extension to RavenDB that allows a developer to take custom action when an event occurs on the server. RavenDB supports the following types of triggers:

TRIGGER	DESCRIPTION	CLASS
Put	Executed when a document (or attachment) is added or modified. The `AllowPut` method can be used for security or validation. `OnPut` can be used to modify document before it is written to disk. Also allows `AfterPut` and `AfterCommit`.	`AbstractPutTrigger` `AbstractAttachmentPutTrigger`

TRIGGER	DESCRIPTION	CLASS
Delete	Executed when a document (or attachment) is deleted. Similar methods to Put trigger such as, `AllowDelete`, `OnDelete`, `AfterDelete`, and `AfterCommit`. `OnDelete` could be used for cascaded deletes (or you can use the cascade delete bundle).	`AbstractDeleteTrigger` `AbstractAttachmentDeleteTrigger`
Read	Executed when a document (or attachment) is read. `AllowRead` can be used for security and `OnRead` can be used to manipulate the contents of the document or metadata before it is returned.	`AbstractReadTrigger` `AbstractAttachmentReadTrigger`
Index query	Allows queries to be modified before they are executed against an index.	`AbstractIndexQueryTrigger`
Index update	Allows custom actions to be taken every time an index entry has been created or deleted.	`AbstractIndexUpdateTrigger` `AbstractIndexUpdateTriggerBatcher`

As you can see, triggers can do almost anything. They are far more powerful than the triggers available in an RDBMS like SQL Server. However, you need to be careful because these triggers execute on the database server and can have adverse implications on performance.

The following example shows how you can use an index update trigger to queue a notification when a document is available. This could be accomplished with a put trigger as well, but by putting it on the index update; we can delay notification until the document is available for searching. This distinction really only matters when the server is under heavy write load since this could delay indexing.

First, descend from the `AbstractIndexUpdateTrigger` class. We only want to fire this extension for indexes containing the text `"Notification"`.

```
public class IndexAlertTrigger : AbstractIndexUpdateTrigger
{
```

```
        public override AbstractIndexUpdateTriggerBatcher
        CreateBatcher(string indexName)
        {
            if(string.Equals(indexName, "Notification", false))
                return new IndexAlertTriggerBatcher(Database);
            else
                return null;
        }
    }
```

Next, descend from the `AbstractIndexUpdateTriggerBatcher` class. This class does the actual work. The `OnIndexEntryCreated` method is called for each document that is placed into the index. This is not just called for new documents but is also called for modified documents. Since the trigger is called with modified documents, the notification example needs to keep track of which documents have received new page notifications.

The `NotificationSent` document is used to track notifications:

```
    public class Notification
    {
        public DateTime QueuedAt { get; set; }
    }
```

The `AbstractIndexUpdateTriggerBatcher` is passed a reference to the database where the trigger fired. This allows the class to read and write other documents from the database. When a document is updated in the index, the trigger first checks if there is a related notification document that has already been created. If not, it creates a `NotificationSent` document to queue notifications to be sent.

```
    public class IndexAlertTriggerBatcher:
                    AbstractIndexUpdateTriggerBatcher
    {
        DocumentDatabase _Database;
        public IndexAlertTriggerBatcher(DocumentDatabase Database)
        {
            this._Database = Database;
        }
        public override void OnIndexEntryCreated(string entryKey,
        Lucene.Net.Documents.Document document)
        {
            var notificationId = "notification/" + entryKey;
            if (_Database.Get(notificationId, null) == null)
            {
```

```
                _Database.Put(notificationId, null,
                RavenJObject.FromObject(new Notification() {
                    QueuedAt = DateTime.Now
                }
                ),
        new RavenJObject(), null);
               }
               base.OnIndexEntryCreated(entryKey, document);
        }
    }
```

Since the code is a part of the indexing process, it has the potential to slow down the indexing of the documents. Careful consideration should be given to the potential side effects of implementing an index trigger. For example, this trigger queues the notifications instead of sending them directly to the SMTP server.

Customizing the storage format

Codecs are an extension that allow for the storage format to be modified. The primary use of codecs is for encryption and compression. There are two basic types of codecs: document and index.

CODEC TYPE	DESCRIPTION	CLASS
Document	Allows the document storage format to be transformed. Encode and Decode methods are implemented for writing and reading data. Document codecs can be used for things like compression or encryption.	`AbstractDocumentCodec`
Index	Allows the index storage format to be transformed. Works just like the document codec, but for indexes.	`AbstractIndexCodec`

Implementing a codec involves processing the stream of data being written from the RavenDB server. It should be implemented in a stream-based or buffer-based manner and not read data into one large byte array. This will keep the memory footprint of the codec low. Since a codec works at the storage layer, it can corrupt data which is not written properly. Only write your own codecs if you really know what you are doing! The following example shows how to implement a GZip-based compression codec:

```
public class GZipCompressionCodec : AbstractDocumentCodec
{
```

```
    public override Stream Encode(string key, RavenJObject data,
  RavenJObject metadata, Stream dataStream)
    {
        return new GZipStream
   (dataStream, CompressionMode.Compress);
    }
    public override Stream Decode(string key,
  RavenJObject metadata, Stream dataStream)
    {
        return new GZipStream
  (dataStream, CompressionMode.Decompress);
    }
}
```

The next example shows how to implement an AES-based encryption codec. Be sure to store the *password* in a secure place and do not lose it! This class only encrypts the document storage so the related `AbstractIndexCodec` class would be needed if the indexes must be encrypted as well:

```
public class AesEncryptionCodec : AbstractDocumentCodec
{
    const string PWD = "password";
    static readonly AesManaged _AesManaged;
    static readonly ICryptoTransform _Encryptor;
    static readonly ICryptoTransform _Decryptor;
    static AesEncryptionCodec()
    {
        using (var derivedkey = new Rfc2898DeriveBytes(PWD, 32))
        {
            var keyBytes = derivedkey.GetBytes(32);
            var ivBytes = derivedkey.GetBytes(16);
            _AesManaged = new AesManaged();
            _Encryptor = _AesManaged.CreateEncryptor
            (keyBytes, ivBytes);
            _Decryptor = _AesManaged.CreateDecryptor
            (keyBytes, ivBytes);
        }
    }

    public override Stream Encode(string key, RavenJObject data,
    RavenJObject metadata, Stream dataStream)
        {
            return new CryptoStream(dataStream, _Encryptor,
    CryptoStreamMode.Write);
        }

    public override Stream Decode(string key,
    RavenJObject metadata, Stream dataStream)
```

```
            {
                return new CryptoStream(dataStream, _Decryptor,
                CryptoStreamMode.Read);
            }
    }
```

This is only a sample code. It is recommended to use the compression & encryption bundles that are provided with RavenDB.

Server-side tasks

Deploying tasks that run off cycle from the document process can also be used to extend the RavenDB server. These tasks can be used for various administrative purposes or to add entirely new functionalities to the server. The tasks are as follows:

TASK TYPE	DESCRIPTION	INTERFACE/CLASS
Server startup	Executes when the server first starts.	`IServerStartupTask`
Database initialization	Executes when a database is initialized. Use when you need something to run once per database. Use `AbstractBackgroundTask` for recurring execution.	`IStartupTask`
Background periodic task	Allows tasks to run on a periodic basis. The `HandleWork` method is executed whenever there is a change on the server such as a document being added or an index is updated. To run at a specified interval, override `TimeoutForNextWork` to specify how often a task should run and return false from the `HandleWork` method once your work is complete.	`AbstractBackgroundTask`

In addition to implementing these interfaces/abstract classes, `IDisposable` should also be implemented to support cleanup functions when the server shuts down.

Summary

RavenDB was architected from the ground up with extensibility in mind. By leveraging these extension points, applications can add new capabilities to the RavenDB server. However, since these extensions are running directly on the RavenDB server, great care should be taken to not adversely affect performance or corrupt data coming from the client. In the final chapter, we'll look at some unique features of RavenDB that really allow you to optimize the user's experience with your application.

9
Optimizing the User Experience

RavenDB makes it easy to give your users great experiences that are often a challenge in traditional relational database systems. Users have come to expect rich experiences from using sites like Google and Amazon. In this chapter, we will cover faceted search, full text search, suggestions, results highlighting, and real-time notifications. These tools will help you take your applications to the next level!

Searching like Amazon

Faceted search give users guided navigation, which allows them to dynamically filter their results based on categories and ranges. These categories or ranges will also show how many matching items they contain. Often, these will be presented on the left side of a set of search results. Getting facets out of RavenDB is as easy as calling the `ToFacets()` method but there is a bit of setup involved to make this work. Perform the following steps:

1. We'll start with a document representing a book title:

```
public class Book
{
    public string Id { get; set; }
    public string Title { get; set; }
    public int Published { get; set; }
    public string Category { get; set; }
    public decimal Cost { get; set; }
    public string Text { get; set; }
}
```

2. Next, we'll set up a basic index for searching:

```
public class BookIndex : AbstractIndexCreationTask<Book>
{
    public BookIndex()
    {
        Map = books => from book in books
        select new
            {
            book.Category,
            book.Cost,
            book.Published,
            book.Title
            };
    }
}
```

3. The next step is to tell RavenDB about the facets. You can define different sets of facets. Each is represented by a special document. The document lists the fields that should be grouped into facets. A field can be grouped by distinct values or into ranges.

```
var facets = new List<Facet> {
    new Facet<Book>
        {
            Name = x => x.Category
        },
    new Facet<Book>
        {
            Name = x => x.Published
        },
    new Facet<Book>
        {
            Name = x => x.Cost,
            Ranges = {
                x => x.Cost<20.0m,
                x => x.Cost>=20.0m && x.Cost<40.0m,
                x => x.Cost>=40.0m && x.Cost<60.0m,
                x => x.Cost>=60.0m && x.Cost<80.0m,
                x => x.Cost>=80.0m
            }
        }
};
```

4. Once the document is created, store it under the `facets` path:

```
RavenSession.Store(new FacetSetup {
Id = "facets/BookFacets",
Facets = facets });
```

5. Now, the facet is ready for use. Here is an example of an MVC controller that queries both the facets and the document results:

```
public ActionResult Index()
{
    var facetResults =
    RavenSession.Query<Book>("BookIndex")
        .ToFacets("facets/BookFacets");
    var bookResults = RavenSession.Query<Book>("BookIndex")
        .OrderBy(x => x.Title)
        .Take(10);
    var results = new BookSearchResults {
         Facets = facetResults,
         Books = bookResults };
    return View(results);
}
```

6. The facets and books are then passed into the view. The facets (especially ranges) don't have a user friendly description, so there is some code in the view to make them easier to understand. Here is the view definition:

```
@foreach (var facet in Model.Facets.Results)
{
    <span>@facet.Key.Replace("_Range",string.Empty):</span>
    foreach (var value in facet.Value.Values)
    {
        if (value.Range.StartsWith("["))
        {
            var limits = value.Range.Split(
            new string[] { "TO", "[", "]", " " },
            StringSplitOptions.RemoveEmptyEntries);
            value.Range =
                ((limits[0] == "NULL") ? "Up " :
                string.Format("{0:C0}",
                Raven.Abstractions.Indexing.
                NumberUtil.StringToNumber(limits[0]))) +
                ((limits[1] == "NULL") ? " and up" :
                string.Format(" to {0:C0}",
                Raven.Abstractions.Indexing.
                NumberUtil.StringToNumber(limits[1])));
        }
```

```
            else
            {
                value.Range =
                TextInfo.ToTitleCase(value.Range);
            }
            <a href="#">@value.Range</a> <span>(@value.Hits)</span>
        }
    }
```

7. We're done! This small amount of code will produce a user friendly screen. Doing the same thing in SQL Server would have been complicated and slow. RavenDB makes it fast and easy!

Book Search Results

Category:
Fantasy (3)
Software Development (5)

Published:
2011 (1)
2012 (5)
2013 (2)

Cost:
Up to $20 (0)
$20 to $40 (4)
$40 to $60 (2)
$60 to $80 (2)
$80 and up (0)

Title	Published	Category	Price
Advanced C# Concepts	2012	Software Development	$49.95
Fly or Die	2012	Fantasy	$39.95
Introduction to the .NET framework	2011	Software Development	$69.95
JSON for monkies	2013	Software Development	$39.95
Maeve Ascendant	2013	Fantasy	$39.95
Midnight Rain	2012	Fantasy	$49.95
Putting the real-time in the web with SignalR	2012	Software Development	$69.95
XML Developer's Guide	2012	Software Development	$39.95

Searching like Google

Performing a full text search is often the easiest way for users to find what they are looking for. However, a great user experience also requires showing highlighted text excerpts and providing spelling suggestions. Since RavenDB is built on Lucene.NET, these tasks become fairly simple. First, our index needs to be updated. It needs to store, index, and vector the text field of our book document:

```
public class BookIndex : AbstractIndexCreationTask<Book>
{
    public BookIndex()
    {
        Map = books => from book in books
```

```
            select new
            {
                book.Category,
                book.Cost,
                book.Published,
                book.Title,
                book.Text
            };
        Store(x => x.Text, FieldStorage.Yes);
        Index(x => x.Text, FieldIndexing.Analyzed);
        TermVector(x => x.Text,
        FieldTermVector.WithPositionsAndOffsets);
        Suggestion(x => x.Text);
    }
}
```

At this point, we can use the `Suggest()` method to get suggestions for misspelled words.

```
var suggestions = MvcApplication.Store.DatabaseCommands.
ForDatabase("wiki").
Suggest("BookIndex", new SuggestionQuery()
{
    Field = "Text",
    Accuracy = 0.4f,
    Distance = StringDistanceTypes.JaroWinkler,
    MaxSuggestions = 4,
    Popularity = true,
    Term = "platfrms"
});
```

But what if we only want to show suggestions if there are no results? RavenDB has a method that can be executed after the query in order to make that happen:

```
var q = RavenSession.Query<Book>("BookIndex").Where(
           x => x.Text == "platfrm");
var first = q.FirstOrDefault();
SuggestionQueryResult suggestions;
if (first == null)
    suggestions = q.Suggest();
```

Now, let's put it all together and add results highlighting. The **MVC** controller will first use the `Customize()` method to tell RavenDB to highlight the matching text and store it in a field called `Highlights`.

```
public ActionResult Search(string text)
{
    var bookResults = RavenSession.Query<Book>("BookIndex")
        .Customize(a => a.Highlight("Text", 128, 10,
        "Highlights"))
        .Where(x => x.Text == text)
        .OrderByScore()
        .Select(x => new Book {
            Title = x.Title,
            Text = x.Text,
            Highlights = x.Highlights
    });
    string[] suggestions = null;
    if (bookResults.FirstOrDefault() == null)
        suggestions = bookResults.Suggest().Suggestions;
    var results = new BookSearchResults {
        Books = bookResults, Suggestions = suggestions
    };

    return View(results);
}
```

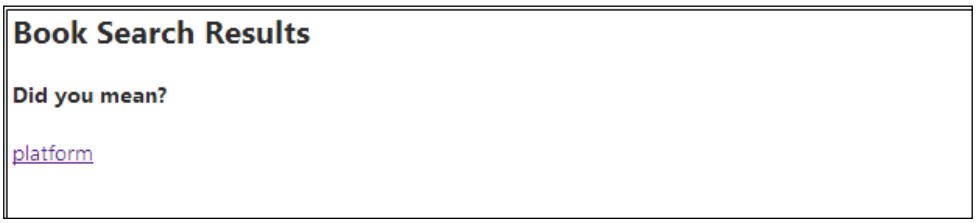
The `Highlights` field gets updated *magically* by the RavenDB runtime but it needs to be part of the select statement. After the query returns, we check to see if there are results. If there aren't any, the suggestions are retrieved and stored in the Model.

The View then uses the Model to display the suggestions (if applicable). Each suggestion is a link that sends the suggestion back into the query as a new search.

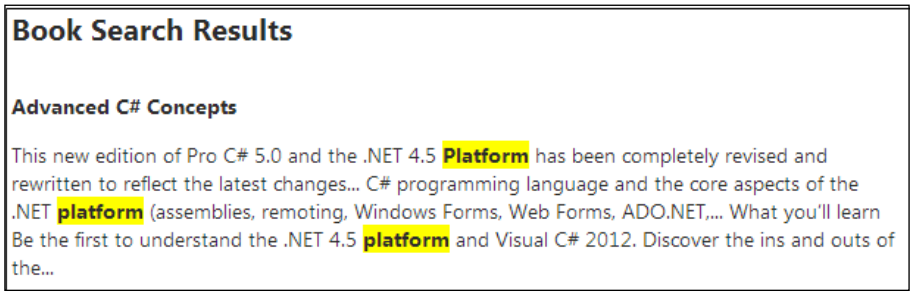
```
@model RavenDB_FacetSearch.Models.BookTextSearchResults
@{
    ViewBag.Title = "Search Results";
}
<h2>Book Search Results</h2>
<div>
    @if (Model.Suggestions!=null)
    {
        <h4>Did you mean?</h4>
        foreach (var suggestion in Model.Suggestions)
        {
```

```
                @Html.ActionLink(suggestion, "Search",
                new { text=suggestion })
                <br />
            }
        }
        <div id="column2">
            @foreach (var book in Model.Books)
            {
                <p><strong>@book.Title</strong></p>
                <p>
                @foreach (var entry in book.Highlights)
                {
                    <span>@Html.Raw(entry)</span><span>...</span>
                }
                </p>
            }
        </div>
    </div>
```

When there are no results, the View will display the suggestion as follows:



However, when results are returned, it displays excerpts from the text with the search text highlighted as in the following screenshot:



Building responsive applications

In the early days of the web, it was enough to present a static web page where hyperlinks provided an *interactive* experience. Today, users have much higher expectations. Fortunately, our tooling has been improving quickly. This makes it much easier to make the web feel responsive.

One important aspect of responsive design is providing user with updated information without forcing them to request for it. This improves the user experience since they are not unknowingly working with stale data.

Stale data is a fact of life when building distributed systems. As soon as a user accesses a web page, another user could have modified the data used to render the page. In some cases, this fact does not really matter. For years, architects have leveraged static or even stale data to improve application scalability. They have done this by performing the following:

- Caching data at different levels of the application stack
- Pre-aggregating results and then refresh them on a scheduled interval
- Pre-render results so the load can be handled overtime instead of being processed when a spike of users hit the site

However, sometimes it really does matter! This is especially the case when users are editing a shared data repository, or taking action based on the data they are viewing. Traditionally, systems have been built to give users an error at the last possible moment before the change takes effect. In this case, concurrency is handled at the data access layer. Just before a row is written to the database, it is checked against the existing row to determine if another user has already modified the data. While this works, it gives the user an awful experience. The user has gone through the trouble of making all their changes and now sees an error when saving, telling them that they weren't even working on the latest copy!

Displaying real-time updates

RavenDB provides an easy to use facility called the **Changes API** to help you solve this problem. Couple this with a technology like **SignalR** and you will have a very nice solution. SignalR is a framework from Microsoft that makes it easy to build real-time communication into your web applications.

The easiest way to look at how these frameworks can work together is through an example. While the user is on the page of our hypothetical wiki, we want them to know if another user has modified the page they are currently viewing. To do this, we need to know that a document representing the page has changed in the database and push this notification all the way to the browser.

SignalR provides communication between clients (browsers) with the server through a **Hub**. The hub is an application-defined API that handles all of the hard work across the wire. In our `MvcApplication.Application_Start` (in `global.asax.cs`), we need to add a line to register the hubs with the SignalR runtime:

```
protected void Application_Start()
{
    Store = new DocumentStore() { Url="http://server:8081" };
    Store.Initialize();
    RouteTable.Routes.MapHubs();
    // additional MVC initialization would go here
}
```

Calling `MapHubs` allows SignalR to provide an API in JavaScript for the browser. In this example, we have a very simple hub. Hubs can be as simple or as complex as per your application's needs. The server side of the hub definition provides the API to the client allowing the client to send messages to the server. The web page will define the client side of the hub definition through JavaScript.

The `PageHub` class allows the browser to tell the server it wants to be notified about the changes made to a given document. In SignalR, Groups can be used to scope message broadcasts to a subset of the clients. The `WatchDocument` method adds the caller to a Group named after the `documentID` function. When the server is ready to send a message to the clients, it can do this only to the ones that are interested in this page:

```
public class PageHub : Hub
{
    public void WatchDocument(string documentID)
    {
        Groups.Add(Context.ConnectionId, documentID);
    }
}
```

Next, let's look at the MVC code. There is a model to define our wiki page:

```
public class Page
{
    public string Id { get; set; }
    public string Title { get; set; }
    public string Content { get; set; }
}
```

The controller and view are where all the interesting things take place. The controller uses the RavenDB Changes API to subscribe to the document changes based on ID. The API also supports filters, such as `ForAllDocuments` and `ForDocumentsStartingWith` as well as subscribing to Index notifications. To make it easier to subscribe for methods, you should include the **Reactive Extensions** package (NuGet package Rx-Main) in your project. This is a library written by Microsoft that makes it easy to write asynchronous and event-based programs. This controller begins by loading a page from the database:

```
public class PageController : BaseController
{
    public ActionResult Index(int id)
    {
        var pageId = "page/" + id;
        var page = RavenSession.Load<Page>(pageId);
```

Next, the controller needs to make sure any prior notification subscriptions are cleaned up. Without this step, each page view will add more and more active subscriptions.

```
    if (Session["PageSubscription"] is IDisposable)
    (Session["PageSubscription"] as IDisposable).Dispose();
```

Finally, using a lambda function, the `PageHub` class is wired up to RavenDB to invoke the `DocumentChanged` method when changes occur.

```
    Session["PageSubscription"] =
    MvcApplication.Store.Changes().ForDocument(pageId).
        Subscribe(changes => {
          var context = GlobalHost.ConnectionManager.
          GetHubContext<PageHub>();
          context.Clients.Group(pageId).DocumentChanged();
        } );
    return View(page);
    }
}
```

`GlobalHost` is a class provided by SignalR for communicating with a hub from outside of a client request context. It retrieves the `PageHub` class and invokes the `DocumentChanged` method on each of the clients that are within the Group named after the `documentID` function that was changed. So where is our `DocumentChanged` method defined? On the client! The view includes JavaScript to define a function called `documentChanged` on the client that can receive messages. Notice the cases where the names are different. SignalR will magically translate the call based on the naming conventions between C# and JavaScript:

```
@model RavenDB_SignalR.Models.Page

<h2>@Model.Title</h2>
<span id="statusmsg" style="display:none">Another user has
modified this page. [<a href="javascript:window.location.
reload(true);">Refresh</a>]</span>

@section scripts {
    <!--Script references. -->
    <script src="/Scripts/jquery-1.7.1.min.js" ></script>
    <script src="/Scripts/jquery.signalR-1.0.1.js"></script>
    <!--Reference the autogenerated SignalR hub script. -->
    <script src="/signalr/hubs"></script>

    <!--Add script to update the page and send messages.-->
    <script type="text/javascript">
        $(function () {
        // Declare a proxy to reference the hub.
            var pageHub = $.connection.pageHub;

            // Create a function that the hub can call
            pageHub.client.documentChanged = function () {
                $('#statusmsg').show();
            };

            // Start the connection.
            $.connection.hub.start().done(function () {
                $.connection.pageHub.server.watchDocument
  ("@Model.Id");
            });
        });
    </script>
}
```

When the page starts and the client can connect to the server, it calls `watchDocument` to register it into the group receiving messages about this document. The server then calls the `documentChanged` function through the SignalR runtime when the change occurs. The `documentChanged` function uses jQuery to notify the user that the page they are viewing has changed by showing the `statusmsg` span. This all happens in real-time as soon as the document changes on the database server!

Summary

Nothing is worse than limiting your user experience by technology choices. RavenDB provides a set of advanced APIs that make building rich user experiences easy. By optimizing these use cases, RavenDB makes it easy for you to create high performance applications that are also high function.

Index

Symbols

__document_id field 22
.NET CLR 45
.NET platform
  document database, using 8
  RavenDB 8

A

AbstractAttachmentDeleteTrigger class 82
AbstractAttachmentPutTrigger class 81
AbstractAttachmentReadTrigger class 82
AbstractBackgroundTask class 86
AbstractDeleteTrigger class 82
AbstractDocumentCodec class 84
AbstractIndexCodec class 84
AbstractIndexQueryTrigger class 82
AbstractIndexUpdateTriggerBatcher class 82
AbstractIndexUpdateTrigger class 82
AbstractPutTrigger class 81
AbstractReadTrigger class 82
aggregate root 14
aggregates 14
aggressive caching 39
AllowReadsFromSecondariesAndWrites ToSecondaries option 69
AllowReadsFromSecondaries option 69
Amazon
  about 89
  faceted search 89-92
Amazon EC2 76
AppHarbor 79
AppHarbor Integration 79
architectural transitions 6
ASP.NET MVC profiler
  enabling 42

B

BitBucket 79
blind sharding 52, 53
bulk insert API
  about 29
  limitations 30
BulkInsertOptions parameter 30

C

Changes API 96
cloud
  about 75
  considerations 75
CloudBird 77
codecs
  about 84
  document 84
  index 84
CodePlex 79
compression bundle 48
configuration, replication 65, 66
configuration settings
  Esent/CacheSizeMax 50
  Esent/DbExtensionSize 50
  Esent/MaxVerPages 50
  MaxNumberOfItemsToIndexInSingleBatch 50
  MaxNumberOfParallelIndexTasks 50
  MemoryCacheExpiration 50
  MemoryCacheLimitMegabytes 50
  MemoryCacheLimitPercentage 50

TransactionMode 50
tuning 50
ConflictException 68
conflicts
about 67
handling 67, 68
handling, automatically 68
Customize() method 94

D

DaaS 76
data
aggregating, reduce used 24
growing 5
storing 26-28
database
document databases 7, 8
rethinking 7
Database as a Service. *See* **DaaS**
database statistics
accessing 46
data changes
handling, over time 25, 26
data model
optimizing 13
data modeling
example 18, 19
DDD 14
delete trigger 82
Distributed Transaction Coordiator (DTC) 30
document
accessing 19-21
loading 24, 25
modeling 13, 14
documentChanged function 99
document codecs 84
document database
about 7, 8
for .NET platform 8
documents
retrieving 10, 11
searching 10, 11
storing 9
documents, accessing
data aggregation, reduce used 24
indexes, working with 21, 22
materialized views 23
related documents, loading 24, 25
document schema
migrating, patching used 37
Domain Driven Design. *See* **DDD**
dynamic index 22

E

EnsureDatabaseExists() method 57
ESENT 45
ETag 38
eventual consistency 10
ExecuteAllPendingLazyOperations() method 33
Extensible Markup Language (XML) 6

F

faceted search 89-92
FailImmediately option 69
failover clustering
about 69, 70
prerequisites 70, 71
read stripping 71
FireBug 43
full text search 92-95

G

GetAttachment command 27
GitHub 79
Glimpse
about 43
installing 44
securing 44
URL 43
used, for profiling 43
Google
about 89
full text search 92-95

H

high availability, RavenDB
failover clustering 63
replication 63

HTTP caching 38
Hub 97

I

IaaS 76
Include() function 24
index codecs 84
indexes 22
  working with 21, 22
index query trigger 82
index update 82
Infrastructure as a Service. *See* **IaaS**
IServerStartupTask interface 86
IStartupTask interface 86

J

JavaScript Object Notation (JSON) 6

K

keys
  used, for modeling relationships 16, 17

L

Language Integrated Queries. *See* **LINQ**
large data loads
  optimizing 29, 30
large result sets
  reading 31, 32
Lazily() method 32, 33
LINQ 9
LoadByUniqueConstraint extension method 21
LoadDocument command 25, 30, 36
Load() method 21
LoadStartingWith() method 17

M

many-to-many relationships 15, 16
master-master topology 64
master-slave topology 64
materialized views 23
Microsoft Azure 76
Microsoft Azure Infrastructure Services 76
Microsoft Extensibility Framework (MEF) 81
multi-master topology 64
multiple document results
  retrieving 32, 33
MVC controller 94
MySQL 60

N

nested collection tags 35
NLog 47
NLog debug logging
  enabling 47
node failures
  handling 68, 69
NoSQL 7

O

Object relational mapping (ORM) 6
OleDb 60
one-to-many relationships 14, 15
one-to-one relationships 14
OnIndexEntryCreated method 83
optimal performance
  server, tuning for 48
Oracle 60

P

PaaS 76
PageHub class 97
Patch() command 34
patching
  about 33
  and scripting 36
  scenarios 36
  used, for migrating document schema 37
Patching API
  about 33
  working 34, 35
Platform as a Service. *See* **PaaS**
plugins directory 21, 27
PostgreSQL 60
profiling
  about 41
  Glimpse used 43

PutAttachment command 27
put trigger 81

R

RAID 48
RAID1 48
RAID5 48
RAID10 48
Raven 8
RavenDB
about 8, 9, 58, 76
aggressive caching 39
configuration settings, tuning 50
extending 81
features 8, 9
HTTP caching 38
triggers 81-84
URL 25
URL, for documentation 71
RavenDB 2.0 29
RavenDB 2.5 39
RavenDB architecture
components 9
documents, retrieving 10, 11
documents, searching 10, 11
documents, storing 9
RavenDB, in cloud
DaaS 76, 77
IaaS 76
PaaS 76
RavenDB plugin
installing 44
RavenHQ 77, 78
Raven Management Studio 77
Reactive Extensions package 98
ReadFromAllServers option 69
read stripping
about 71
using 72, 73
read trigger 82
real-time updates
displaying 96-99
recovery point object (RPO) 75
recovery time object (RTO) 75
reduce
used, for aggregating data 24
Redundant Array of Inexpensive Drives.
See **RAID**
relational database
about 5
reporting, scaling out to 58-62
relationships
modeling 14
modeling, keys used 16, 17
relationships, modeling
many-to-many relationships 15, 16
one-to-many relationships 14, 15
one-to-one relationships 14
Remove() method 36
replication
about 63
configuring 65, 66
conflicts, handling 67, 68
node failures, handling 68, 69
ReplicationInformer class 72
replication mechanisms
master-master topology 64
master-slave arrangement 64
multi-master topology 64
reporting
scaling out, to relational database 58-62
responsive applications
building 96
real-time updates, displaying 96-99

S

SaaS 6
ScriptedPatchRequest request 36
scripting
and patching 36
script patches 36
semi-structured data 6
server
tuning, for optimal performance 48
server-side tasks 86
server statistics
accessing 46
Service Oriented Architectures (SOA) 6
sharding
about 51, 52
based, on data 54
based, on IShardResolutionStrategy

interface 55-57
blind sharding 52, 53
ShardOn() method 56
ShardResolutionStrategy property 56
SignalR 96
Skip() method 31
Software as a Service. *See* **SaaS 6**
solid state disk (SSD) 48
SQL CE 60
SQL Replication bundle 58
SQL Server 60
Statistics() method 31
Storage Area Network (SAN) 48, 70
storage format
customizing 84-86
Store() method 30
Stream() method 32
Suggest() method 93

T

Take() method 31
task type
background periodic task 86
database initialization 86
server startup 86
Temporal Versioning
URL 26
ToFacets() method 89
triggers
about 81
delete 82
example 83, 84
index query 82
index update 82
put 81
read 82

U

UniqueConstraintsStoreListener listener 21
user experience
optimizing 89

V

VARCHAR(MAX) method 21

W

web-based applications
profiling 41, 42
Wikigalxi 55
Windows Performance Counters
about 45
monitoring with 45

Thank you for buying

RavenDB High Performance

About Packt Publishing

Packt, pronounced 'packed', published its first book "*Mastering phpMyAdmin for Effective MySQL Management*" in April 2004 and subsequently continued to specialize in publishing highly focused books on specific technologies and solutions.

Our books and publications share the experiences of your fellow IT professionals in adapting and customizing today's systems, applications, and frameworks. Our solution based books give you the knowledge and power to customize the software and technologies you're using to get the job done. Packt books are more specific and less general than the IT books you have seen in the past. Our unique business model allows us to bring you more focused information, giving you more of what you need to know, and less of what you don't.

Packt is a modern, yet unique publishing company, which focuses on producing quality, cutting-edge books for communities of developers, administrators, and newbies alike. For more information, please visit our website: `www.packtpub.com`.

About Packt Open Source

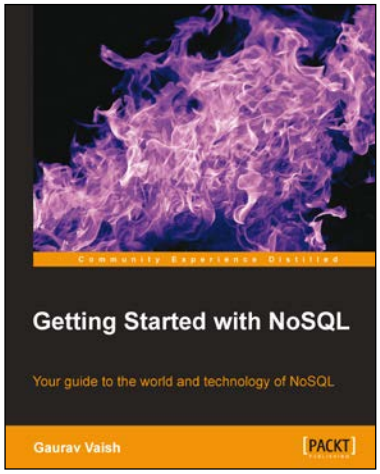
In 2010, Packt launched two new brands, Packt Open Source and Packt Enterprise, in order to continue its focus on specialization. This book is part of the Packt Open Source brand, home to books published on software built around Open Source licences, and offering information to anybody from advanced developers to budding web designers. The Open Source brand also runs Packt's Open Source Royalty Scheme, by which Packt gives a royalty to each Open Source project about whose software a book is sold.

Writing for Packt

We welcome all inquiries from people who are interested in authoring. Book proposals should be sent to author@packtpub.com. If your book idea is still at an early stage and you would like to discuss it first before writing a formal book proposal, contact us; one of our commissioning editors will get in touch with you.

We're not just looking for published authors; if you have strong technical skills but no writing experience, our experienced editors can help you develop a writing career, or simply get some additional reward for your expertise.

[PACKT] PUBLISHING open source
community experience distilled



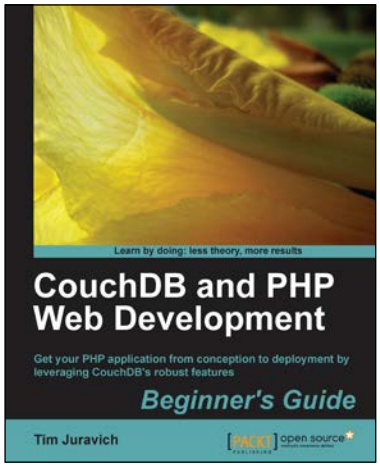
Getting Started with NoSQL

ISBN: 978-1-849694-98-8 Paperback: 142 pages

Your guide to the world and technology of NoSQL

1. First hand, detailed information about NoSQL technology
2. Learn the differences between NoSQL and RDBMS and where each is useful
3. Understand the various data models for NoSQL
4. Compare and contrast some of the popular NoSQL databases on the market

---



CouchDB and PHP Web Development Beginner's Guide

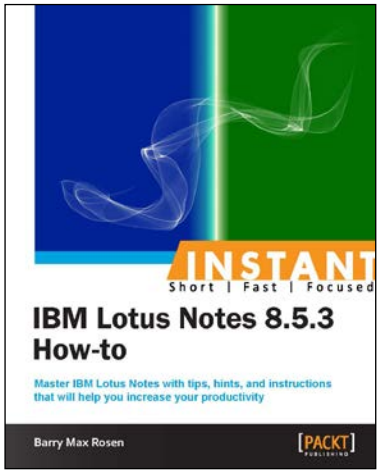
ISBN: 978-1-849513-58-6 Paperback: 304 pages

Get your PHP applicati on from concepti on to deployment by leveraging CouchDB's robust features

1. Build and deploy a flexible Social Networking application using PHP and leveraging key features of CouchDB to do the heavy lifting
2. Explore the features and functionality of CouchDB, by taking a deep look into Documents, Views, Replication, and much more.
3. Conceptualize a lightweight PHP framework from scratch and write code that can easily port to other frameworks

Please check **www.PacktPub.com** for information on our titles

[PACKT] open source
community experience distilled
PUBLISHING

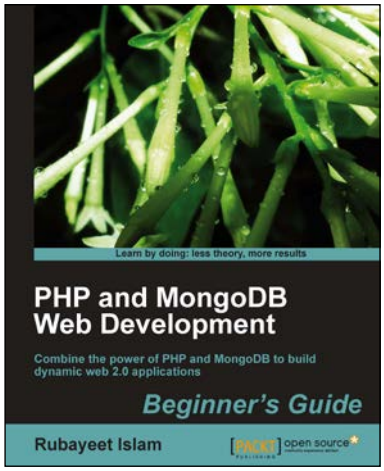


Instant IBM Lotus Notes 8.5.3 How-to

ISBN: 978-1-782170-22-8 Paperback: 68 pages

Master IBM Lotus Notes with tips, hints, and instructions that will help you increase your productivity

1. Learn something new in an Instant! A short, fast, focused guide delivering immediate results.
2. Customize your Lotus Notes 8.5.3 experience
3. Work with Calendars and To do lists
4. Move from Outlook to IBM Lotus Notes 8.5.3



PHP and MongoDB Web Development Beginner's Guide

ISBN: 978-1-849513-62-3 Paperback: 292 pages

Combine the power of PHP and MongoDB to build dynamic web 2.0 applications

1. Learn to build PHP-powered dynamic web applications using MongoDB as the data backend
2. Handle user sessions, store real-time site analytics, build location-aware web apps, and much more, all using MongoDB and PHP
3. Full of step-by-step instructions and practical examples, along with challenges to test and improve your knowledge

Please check **www.PacktPub.com** for information on our titles

www.allitebooks.com